

UniWAM Technical Report: Unified Mobile Manipulation via Mixed-Stream World-Action Modeling and Manipulation Anchor Pose Supervision

**Wei Xue^{*}, Kelian Liu^{*}, Mingzhang Cui, Jinhua Xie, Jianan Hou,
Jingcheng Lu, Lintao Wang, Kaixiang Qiu, Yizhou Liu, Xinghai Ye,
Jinghang Han, Mingcheng Li, Jie Gu, Shunli Wang, Lihua Zhang[§], Dingkang Yang^{†,§}**

Physical Superintelligence Lab, Fysics AI College of Intelligent Robotics and Advanced Manufacturing, Fudan University

^{*}Equal contribution; order decided by a coin flip, [†]Project lead, [§]Corresponding author

Abstract

Mobile manipulation requires precise navigation to a manipulation-ready pose followed by reliable object interaction. These two stages differ in action spaces and visual requirements, which complicates unified policy learning. In addition, collecting diverse real-world navigation data with explicit manipulation-ready pose supervision remains costly and difficult to scale. We introduce UniWAM, a unified mixed-stream world-action model with separate action encoders and output heads for navigation and manipulation, sharing a common backbone. This design supports joint representation learning on independently sampled navigation and manipulation data. UniWAM supports independent inference for either stream and batch-parallel inference for both. We further introduce Manipulation Anchor Pose (MAP) supervision for where to stop and how to orient for manipulation. An automated pipeline constructs MAP-Data from large-scale 3D scenes, yielding over 1.5 million episodes and 7,500 hours. MAP-Data provides per-frame target-object bounding boxes and image-plane MAP coordinates as auxiliary navigation supervision. Together with projected end-effector trajectories for manipulation, these prediction targets provide stream-specific image-plane supervision for action learning from egocentric observations. With large-scale MAP-Data, UniWAM outperforms the strongest external baselines on our MAP-Bench by 30.1% in position error and 44.0% in heading error. Across 24 real-robot tasks, UniWAM achieves leading results in MAP navigation and mobile manipulation, with competitive manipulation performance. We have released code, data, and benchmark.

Date: September 3, 2026

Corresponding: wxue24@m.fudan.edu.cn, klliu25@m.fudan.edu.cn, dkyang20@fudan.edu.cn,
lihuazhang@fudan.edu.cn

Project Page: <https://fysics-ai.github.io/uniwam.github.io/>

Github: <https://github.com/Fysics-AI/UniWAM>

Hugging Face: <https://huggingface.co/Fysics-AI/UniWAM>
<https://huggingface.co/datasets/Fysics-AI/MAP-Data>
<https://huggingface.co/datasets/Fysics-AI/MAP-Bench>

1 Introduction

Mobile manipulation requires a robot to move through an environment and interact precisely with objects after arrival. Compared with navigation or fixed-base manipulation alone, mobile manipulation supports a much broader range of tasks, such as fetching an item from another room or clearing a table in a different part of a house [13, 39]. However, navigation and manipulation differ in both what they need to observe and how they act. Navigation needs a forward view

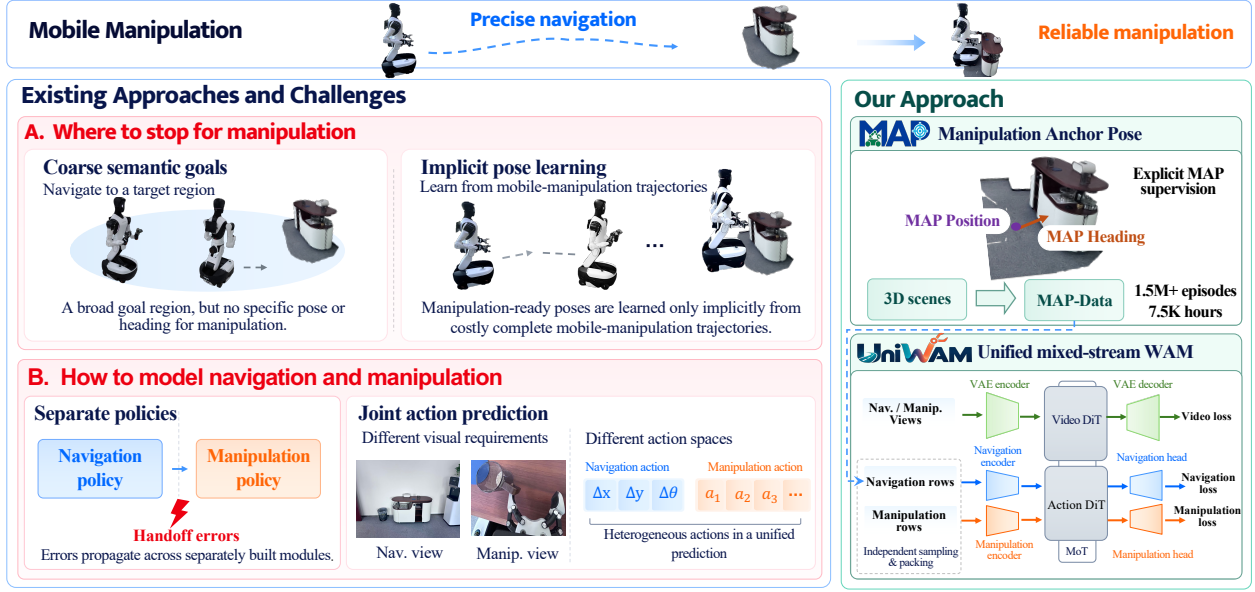


Figure 1 Challenges of mobile manipulation and our approach. Existing unified policies either share one action representation across navigation and manipulation or separate the two only inside the action module, and they learn where to stop only implicitly from costly complete trajectories. UniWAM models navigation and manipulation as separate streams over a shared backbone, and MAP-Data supervises the manipulation-ready pose at scale with automatically generated trajectories.

of the scene ahead, including distant targets and free space, whereas manipulation needs detailed views of the local workspace, often from downward-facing and wrist cameras [7, 35, 48]. Their action signals also differ in frequency. Base motion varies slowly and smoothly over long distances, whereas arm and gripper motion changes rapidly near contact [48]. A single policy for both behaviors must reconcile two visual requirements and two action spaces.

Research on mobile manipulation mainly follows two routes, namely modular systems and unified models. Modular systems train navigation and manipulation policies separately and connect them at execution time [31]. This handoff adds an interface between the two policies and can propagate errors from one module to the next [31, 48]. Unified models instead learn the complete mobile-manipulation process end to end within a single policy. Recent unified approaches build on vision-language-action (VLA) models [2, 23] and world-action models (WAMs) [12, 41] to predict navigation and manipulation actions within a shared model and a shared training pipeline.

Unified mobile manipulation still faces two linked challenges, illustrated in Fig. 1. The first is architectural. A shared action token for navigation and manipulation [2, 3] ignores their heterogeneous visual observations, action spaces, and control rates. Separate action experts [9] decouple the action spaces, yet both designs still jointly supervise both action targets within the same training sample. Because mobile manipulation is largely sequential, the arm is mostly idle during navigation and the base is mostly idle during manipulation. Joint supervision thus spends much of the training on stationary commands, ties the sampling of the two behaviors to complete trajectories, and prevents direct use of navigation-only or manipulation-only data, which are far easier to collect.

The second challenge concerns data and follows from the first. Existing unified models learn the manipulation-ready terminal pose only implicitly from complete trajectories, so they require large-scale mobile-manipulation data, which are costly to collect [33, 47]. Without such data, a policy may reach the target but stop at an unsuitable position or heading, and the subsequent manipulation then fails [36]. Semantic navigation data scale better, but they specify only target proximity, not the terminal pose required for manipulation [15].

To address the first challenge, we propose **UniWAM**, a mixed-stream world-action model with separate navigation and manipulation streams. Each stream has its own visual inputs, action encoder, and output head over shared video and action DiTs. The two streams are sampled independently and packed as separate batch rows without cross-row attention, so each sample supervises only its own stream. Navigation-only, manipulation-only, and mobile-manipulation data can

all be used directly, and either stream runs alone or both run in one batch at deployment.

To address the second challenge, we introduce the **Manipulation Anchor Pose (MAP)**, the base position and heading at which navigation should end for manipulation. Unlike floor affordances, candidate base placements, or pose preferences learned from rollouts [6, 8, 43, 45], MAP is an explicit target that can be generated automatically and supervises the entire approach trajectory. We build **MAP-Data** from large-scale indoor 3D scenes with an automated pipeline, yielding over 1.5 million navigation episodes and 7,500 hours annotated with per-frame target bounding boxes and image-plane MAP coordinates. We hold out a subset of its targets as **MAP-Bench** for evaluating MAP navigation. Because the streams are sampled independently, UniWAM trains its navigation stream directly on MAP-Data, which supervises where to stop for mobile manipulation explicitly and at scale. Together with projected end-effector trajectories for manipulation, the bounding boxes and MAP coordinates serve as image-plane auxiliary targets for the two streams [25, 37].

Our contributions are:

- We propose **UniWAM**, a mixed-stream world-action model that learns navigation and manipulation as two independently sampled streams over a shared backbone.
- We define the **Manipulation Anchor Pose (MAP)** and generate **MAP-Data**, over 1.5M episodes labeled with MAP and target boxes. To our knowledge, it is the largest dataset for manipulation-oriented navigation. We hold out **MAP-Bench**, a leakage-free benchmark for MAP navigation.
- We evaluate UniWAM on MAP-Bench and 24 real-robot tasks. Through mixed training on MAP-Data and diverse manipulation data, UniWAM outperforms pretrained baselines, reducing position and heading errors by 30.1% and 44.0% and raising mobile-manipulation success by 17.5 points.

2 Related Work

Unified mobile-manipulation models. Most unified policies predict base, arm, and gripper commands through a shared action representation. RT-1 adds base motion to arm control and selects the controlled component with a discrete mode variable [5]. Mobile ALOHA concatenates bimanual joint positions with base linear and angular velocities into one action vector [13]. VLA models such as π_0 [4], $\pi_{0.5}$ [3], GR00T [2], and Xiaomi-Robotics-1 [32] follow the same interface. They predict all degrees of freedom in one action chunk and use navigation-only or manipulation-only data only by zero-padding or masking the absent dimensions. This design learns coordinated behavior, but one action token must cover navigation and manipulation despite their different visual inputs, action spaces, and control rates, and each sample from a complete trajectory supervises both behaviors, including the idle one. World-action models (WAMs) build on video generation models to jointly predict future visual states and actions [1, 16, 17, 22, 38, 41]. Most WAMs target fixed-base manipulation. ABot-M0.5 [9] and MobileWAM [12] extend them to mobile manipulation. ABot-M0.5 assigns base and manipulation actions to separate sub-towers within a dual-level mixture-of-transformers. MobileWAM attaches an action expert to a pretrained video diffusion transformer, routes its feed-forward layers across shared, locomotion, and manipulation experts, and supervises future latents with Chain-of-Foresight. Both methods separate navigation and manipulation only within the action module. The two behaviors still share visual inputs, and each sample still supervises both. Cross-embodiment policies such as CrossFormer [11] and HPT [28] use embodiment-specific tokenizers or stems and action heads over a shared trunk, so each sample supervises the action space of one robot. They separate different robots, however, not the navigation and manipulation phases of one mobile manipulator, and they have neither a video world model nor an explicit terminal pose. UniWAM applies this separation within one mobile manipulator. Each stream has its own visual inputs, action encoder, and output head and is sampled independently, while both streams share the video and action DiTs. It further combines this design with a video world model and explicit MAP supervision. Table 1 summarizes these differences across views, sampling, world modeling, and terminal-pose supervision.

Manipulation-oriented navigation. Unlike semantic navigation, which only requires reaching the vicinity of a target [15, 42], manipulation-oriented navigation must find a base position and heading that support the next interaction. Existing methods fall into three groups. The first searches for feasible base placements from arm reachability [26]. The second learns manipulation-ready locations from labeled or executed data, including floor affordance maps [43, 45], grasp-aware base placement [8], and pose preferences learned from manipulation rollouts [6]. The third infers the base pose at test time by optimizing compatibility with a manipulation policy [36] or by reasoning with vision-language models [30, 44]. These methods output candidate poses, affordance regions, or placement scores for a specific robot,

Table 1 Comparison with recent unified mobile-manipulation models and cross-embodiment policies. The first three are VLA policies, the next two are world-action models, and the last two are cross-embodiment policies, which separate different robots rather than the navigation and manipulation phases of one mobile manipulator. Separate views: navigation and manipulation receive different camera inputs. One behavior per sample: a training sample can supervise one behavior alone without padding or masking. Video world model: the model predicts future video. Explicit terminal pose: the terminal base pose for manipulation is supervised directly.

Model	Action generator	Separate views	One behavior per sample	Video world model	Explicit terminal pose
$\pi_{0.5}$ [3]	single action expert for all degrees of freedom	✗	✗	✗	✗
GR00T N1.7 [2]	single action head for all degrees of freedom	✗	✗	✗	✗
Xiaomi-Robotics-1 [32]	single action head for all degrees of freedom	✗	✗	✗	✗
ABot-M0.5 [9]	action stream with mobility and manipulation sub-towers	✗	✗	✓	✗
MobileWAM [12]	action expert with routed locomotion and manipulation experts	✗	✗	✓	✗
CrossFormer [11]	shared transformer with embodiment-specific action heads	✓	✓	✗	✗
HPT [28]	shared trunk with embodiment-specific stems and heads	✓	✓	✗	✗
UniWAM	shared action DiT with separate stream interfaces	✓	✓	✓	✓

Table 2 Comparison of MAP-Data with methods for manipulation-ready base placement. Supervises approach: the labels cover the entire approach trajectory rather than only a terminal pose or a score for a given scene. Scale is the amount of labeled data reported in each paper, and a dash denotes methods that do not learn from a dataset, such as kinematic search or test-time optimization.

Method	Output	Label source	Supervises approach	Scale
Reachability inversion [26]	feasible base poses	arm kinematics	✗	—
MoMa-Kitchen [43]	floor affordance map	simulation	✗	127K episodes
FloAff-Kitchen [45]	floor affordance map	simulation	✗	24.8K observations
GBPP [8]	scores of candidate base poses	rule labels and simulation trials	✗	180K rule and 12K simulation labels
N2M [6]	distribution over base poses	policy rollouts	✗	12 to 15 rollouts per task
Mobi- π [36]	base pose for a given policy	test-time optimization	✗	—
MAP-Data	terminal pose, per-frame box and MAP labels	scene geometry and a VLM	✓	1.5M episodes

policy, or scene. None of them supervises the approach trajectory that leads to the chosen pose. As a result, navigation data that explicitly supervise the manipulation-ready terminal pose along entire approach trajectories remain scarce, although such supervision is essential for learning where to stop. We define this terminal pose as the MAP and generate MAP-Data from large-scale indoor 3D scenes. MAP-Data supervises the entire approach with an explicit terminal pose and dense image-plane labels. The largest prior dataset, MoMa-Kitchen with 127K episodes, is about one-twelfth the size of MAP-Data with its 1.5M episodes. Table 2 compares them in output, label source, and scale.

3 Method

This section first defines the MAP and describes how MAP-Data and MAP-Bench are built (Sec. 3.1). It then presents UniWAM, including its stream interfaces, mixed-stream batch packing, independent stream sampling, image-plane auxiliary supervision, training objective, and implementation (Sec. 3.2).

3.1 MAP: Manipulation Anchor Pose

Existing representations of manipulation-ready base placement are specific to a robot, policy, or scene and do not supervise the approach trajectory [6, 43]. We instead make the terminal base pose an explicit target and generate approach trajectories toward it at scale, which yields dense supervision for every step of the approach.

MAP definition. For a manipulation target o_i , its *Manipulation Anchor Pose* (MAP) is $\mathbf{m}_i = (\mathbf{p}_i^{\text{MAP}}, \theta_i^{\text{MAP}})$, where $\mathbf{p}_i^{\text{MAP}} \in \mathbb{R}^2$ and $\theta_i^{\text{MAP}} \in \mathbb{S}^1$ denote the base position and heading. The MAP explicitly specifies where the robot should stop and how it should orient for manipulation. Because it is defined by scene geometry rather than by a particular manipulation policy, it can be computed automatically for any target in a 3D scene.

Scene sources and target curation. MAP-Data is built from HM3D and HM3D-Semantics [24, 34], InteriorGS [20], and a large in-house scene corpus. As illustrated in Fig. 2, the pipeline has three stages. Stage S1 converts these heterogeneous

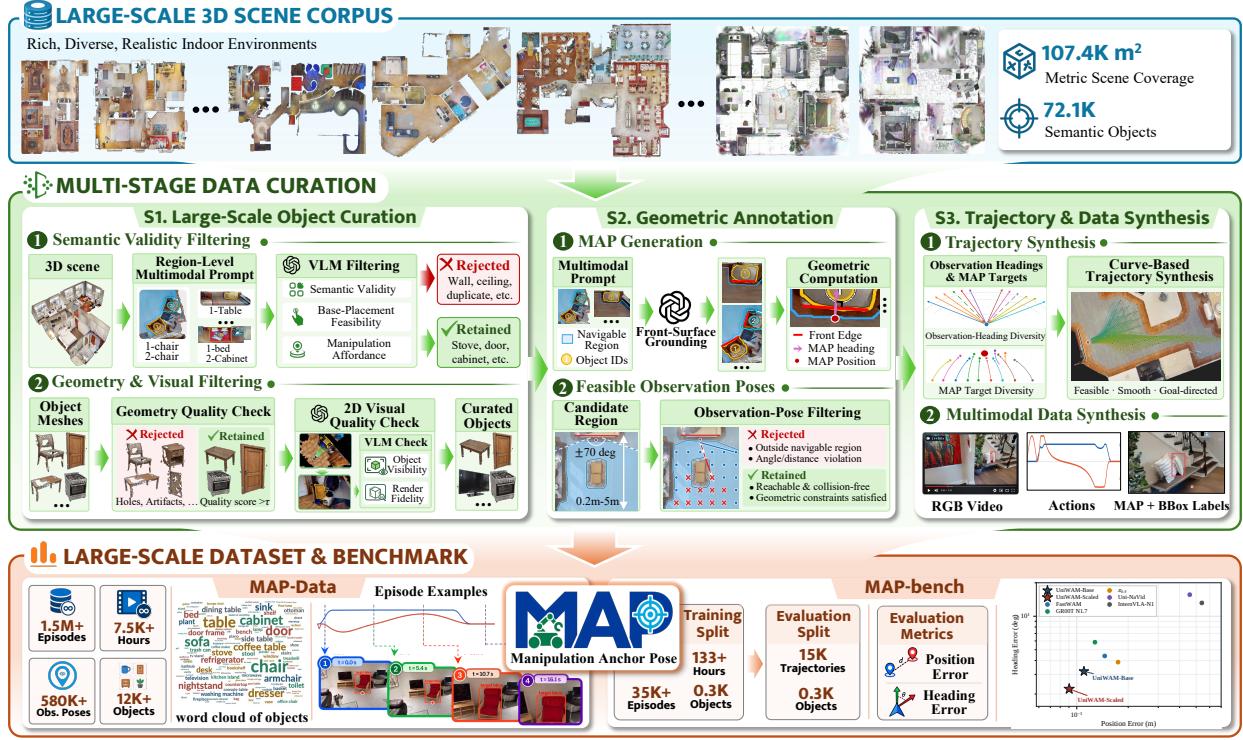


Figure 2 Overview of MAP-Data and MAP-Bench. From large-scale indoor 3D scenes, S1 curates valid target objects, S2 annotates each target with a MAP and feasible observation poses, and S3 synthesizes approach trajectories with RGB video, actions, and MAP and box labels. MAP-Bench evaluates position and heading errors from held-out initial poses.

sources into a common representation of target instances, navigable regions, and renderable observations. Mesh-based scenes undergo geometry and navigability checks, whereas Gaussian-splatting scenes are validated through occupancy maps and rendered observations. For sources without object-instance annotations, semantic instance segmentation supplies candidate target masks. Candidate targets then pass four filters. Semantic filtering verifies that the instance matches the task condition and is uniquely identifiable. Geometry filtering rejects incomplete target surfaces and corrupted scene regions. Base-placement filtering requires collision-free navigable space near the manipulation-facing surface. Visibility and rendering filtering removes targets that cannot be observed reliably from feasible camera poses or that produce invalid rendered views, and the remaining targets proceed to MAP generation.

MAP and observation-pose generation. Stage S2 constructs the MAP of each target and a set of feasible observation poses. A vision-language model identifies the manipulation-facing front edge of the target. Let $\mathbf{p}_f$ be the ground-plane midpoint of this edge and $\mathbf{n}_f$ its unit normal pointing toward navigable space. Denoting the clearance-aware navigable region by $\mathcal{F}_{\text{nav}}$, the MAP position and heading are

$$\mathbf{p}^{\text{MAP}} = \text{Ray}(\mathbf{p}_f, \mathbf{n}_f) \cap \partial\mathcal{F}_{\text{nav}}, \quad \theta^{\text{MAP}} = \text{atan2}\left((\mathbf{p}_f - \mathbf{p}^{\text{MAP}})_y, (\mathbf{p}_f - \mathbf{p}^{\text{MAP}})_x\right). \quad (1)$$

The intersection of the ray with the boundary of the navigable region determines where the base stops, and the heading makes the robot face the target surface. Observation poses are sampled independently across target distance, bearing, and camera heading. They are retained only when the base lies in navigable free space, satisfies collision and geometric constraints, and provides a valid view of the target. Sampling observation poses separately from the MAP diversifies the initial states while keeping the terminal target fixed for all trajectories of a target.

Trajectory synthesis. Stage S3 connects each retained observation pose to its MAP by a single-segment or multi-segment cubic Bézier trajectory. A segment with control points $\{\mathbf{c}_j\}_{j=0}^3$ is

$$\mathbf{q}(s) = (1-s)^3\mathbf{c}_0 + 3(1-s)^2s\mathbf{c}_1 + 3(1-s)s^2\mathbf{c}_2 + s^3\mathbf{c}_3, \quad s \in [0, 1]. \quad (2)$$

The control points preserve the initial motion direction and terminate at the MAP with the prescribed heading. If a direct curve is obstructed, A* [14] provides a collision-free corridor that guides piecewise fitting. A trajectory is retained only if it remains within the clearance-aware navigation mask, approaches the target from the annotated side, keeps the manipulation-facing boundary within the horizontal field of view, and satisfies curvature and path-length constraints. Trajectories with excessive backtracking or an inaccurate terminal position or heading are rejected.

Synchronized geometric labels. At each trajectory step, the renderer records the egocentric RGB observation and the planar navigation action. The MAP position is projected into the current camera as

$$\mathbf{u}_t^{\text{MAP}} = \Pi \left(T_{c_t \leftarrow w} \begin{bmatrix} x_{\text{MAP}} & y_{\text{MAP}} & z_{\text{MAP}} & 1 \end{bmatrix}^\top \right), \quad (3)$$

where $T_{c_t \leftarrow w}$ maps world coordinates to the camera frame and Π applies the camera intrinsics and perspective division. Target-instance masks are converted to image-plane bounding boxes. RGB frames, navigation actions, projected MAP coordinates, target boxes, scene and target identifiers, and quality flags are stored with the same timestamps for frame-level alignment. Each trajectory also carries five target-specific instructions. MAP-Data contains over **1.5M** episodes and **7.5K** hours of navigation trajectories with explicit MAP and image-plane geometric supervision.

MAP-Bench. MAP-Bench contains 300 targets in 109 scenes. Its training split contains 35,454 trajectories from 6,732 observation poses, with 7,184,098 RGB frames at 15 Hz, and its evaluation split contains 15K trajectories. Each target has five equivalent English instructions, and one of them is sampled for each episode with a fixed random seed. Each episode specifies a reference MAP and an initial pose. Episodes are grouped by their initial distance to the reference MAP into a short range below 1.5 m, a mid range from 1.5 to 3.5 m, and a long range of at least 3.5 m. To prevent leakage between the splits, we cluster feasible initial poses in $SE(2)$ with the normalized distance

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\left(\frac{\|\mathbf{p}_i - \mathbf{p}_j\|_2}{r_p} \right)^2 + \left(\frac{\text{wrap}(\theta_i - \theta_j)}{r_\theta} \right)^2}, \quad (4)$$

where r_p and r_θ set the spatial and angular clustering scales. Each cluster is assigned entirely to one split. The two splits therefore share targets but never share an initial pose or a near-duplicate initial view, so the evaluation measures generalization to unseen starting configurations rather than memorized trajectories. All MAP-Bench targets are further excluded from the rest of MAP-Data, so the additional data used by UniWAM-Scaled contain no benchmark target, although MAP-Data may contain other targets in the benchmark scenes. Sec. 4.3 evaluates unseen scenes and targets zero-shot on real robots, which complements the held-out initial poses of MAP-Bench.

Metrics and evaluation. We evaluate MAP navigation with the MAP Position Error and the MAP Heading Error. For N evaluation trajectories, they are

$$E_{\text{pos}} = \frac{1}{N} \sum_{r=1}^N \|\hat{\mathbf{p}}_r - \mathbf{p}_r^{\text{MAP}}\|_2, \quad E_{\text{head}} = \frac{1}{N} \sum_{r=1}^N \left| \text{wrap}(\hat{\theta}_r - \theta_r^{\text{MAP}}) \right|, \quad (5)$$

where $\hat{\mathbf{p}}_r$ and $\hat{\theta}_r$ are the reached position and heading, and $\mathbf{p}_r^{\text{MAP}}$ and θ_r^{MAP} are those of the reference MAP. Eq. 5 is applied per distance range, and Overall averages the three ranges. All models are evaluated in closed loop from the same held-out initial poses with the same instructions. A predicted action chunk is regarded as near zero if at least 80% of its predicted relative poses, including the last one, satisfy $\|(x_t, y_t)\|_2 \leq 0.05$ m and $|\theta_t| \leq 0.05$ rad. For models that predict velocities, the test is applied to the base displacement integrated over the chunk. An episode stops after three consecutive near-zero chunks, and the third chunk is not executed. Uni-NaVid stops when it predicts its stop token. An episode also ends after twice the steps of its reference trajectory, which bounds the evaluation time of models that never stop.

3.2 UniWAM: Unified Mixed-Stream World-Action Model

UniWAM separates navigation and manipulation at the level of control streams rather than within the action module, as shown in Fig. 3. Each stream has its own visual inputs, action encoder, output head, and learnable tokens. Both streams share a video DiT, initialized from Wan2.2-Ti2V-5B [27], and an action DiT, coupled through mixture-of-transformers attention [18]. This design addresses the two problems identified in Sec. 1. Stream-specific interfaces accommodate the heterogeneous observations and action spaces, and independent sampling lets each training sample supervise only one stream. The shared DiTs keep a single model and a shared representation for both streams.

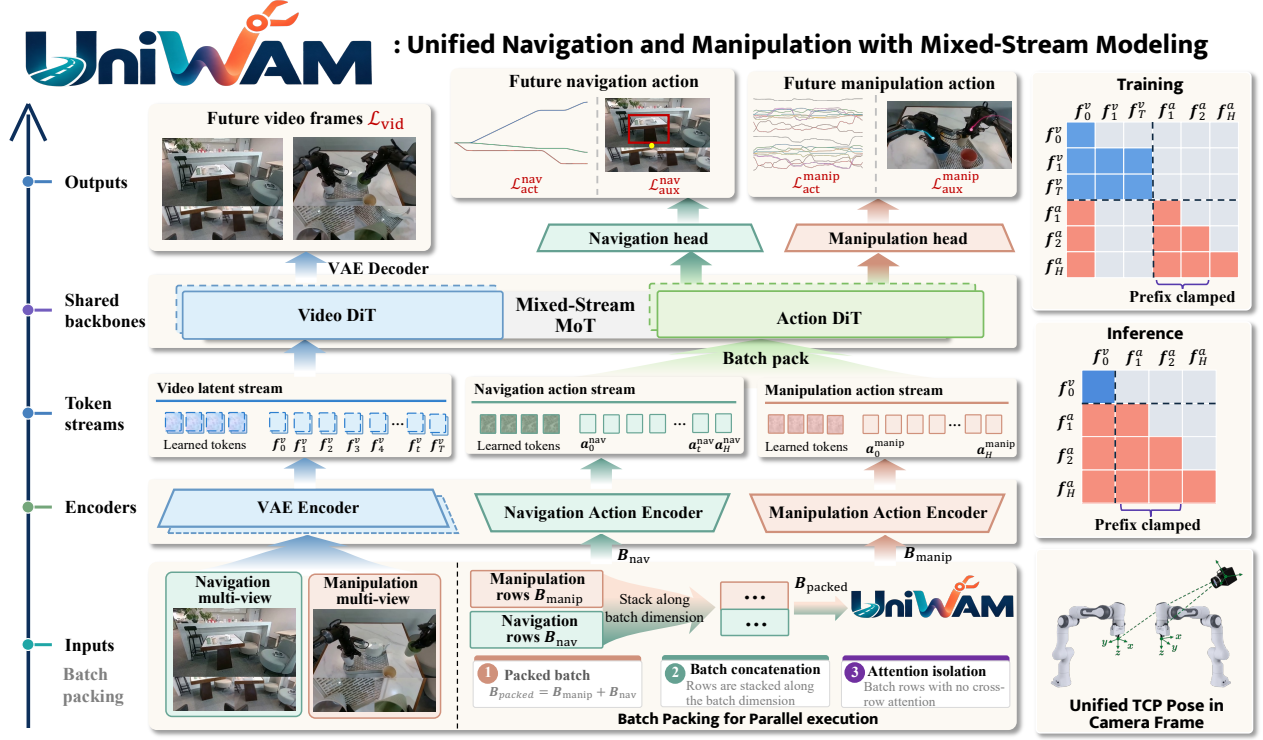


Figure 3 Overview of UniWAM. Navigation and manipulation samples are drawn independently, encoded by stream-specific action encoders, and packed as separate batch rows through the shared video and action DiTs. Stream-specific heads predict actions and image-plane auxiliary targets, while the video branch predicts future frames. Right: attention masks with clean action prefixes for training and inference, and the camera-frame end-effector pose shared across embodiments.

3.2.1 Stream Interfaces and Action Representations

Let $s \in \{\text{nav}, \text{manip}\}$. At time t , stream s receives camera views o_t^s , the instruction ℓ , and the proprioceptive state q_t , and predicts an action chunk

$$A_t^s = (a_{t,1}^s, \dots, a_{t,H}^s) = f_\phi^s(o_t^s, \ell, q_t), \quad (6)$$

where H is the action horizon, a hyperparameter, and f_ϕ^s shares all DiT parameters ϕ across streams. A stream-specific linear encoder maps each action to the shared hidden space, and a stream-specific head decodes it. Because the DiTs are shared, learnable tokens for the video and actions of each stream mark the view and action space of each sequence.

Navigation actions. Many unified policies represent base motion by linear and angular velocities (v, ω) , for example Mobile ALOHA [13]. UniWAM instead predicts planar pose increments $(\Delta x, \Delta y, \Delta \theta)$ relative to the base pose at the start of the action chunk, and a chunk is a sequence of such waypoints over the horizon. This choice addresses a property of base velocity labels. Mobile bases run under conservative velocity limits with brief acceleration phases, so recorded linear and angular velocities lie mostly either near zero or at the velocity limits. As a result, the velocity labels follow a strongly bimodal distribution, much like the binary open and close states of a gripper. Neural networks learn continuous, smoothly varying targets more easily than such bimodal ones, which are known to be hard to fit with a continuous action model [10]. Converting base motion into pose increments turns the bimodal velocity labels into continuous values, because the increments accumulate the velocities over the chunk and vary smoothly along an approach trajectory. Pose increments also remain easy to execute, because a low-level controller converts them into target velocities for the base. The ablation in Sec. 4.5 confirms this design, especially for long approaches.

Manipulation actions. Each arm is represented by 10 dimensions, namely a 3D end-effector position, a 6D rotation, and a 1D gripper state, so the dual-arm action has $(3 + 6 + 1) \times 2 = 20$ dimensions. The 6D rotation consists of the first two columns of the rotation matrix, and the full matrix is recovered by Gram-Schmidt orthogonalization [46]. Unlike Euler angles and quaternions, this representation is continuous, so nearby rotations have nearby targets. Positions and

rotations are expressed in the current main camera frame, which makes visually similar motions numerically similar across embodiments [40]. For single-arm robots, the absent arm is zero-padded and masked from the loss.

3.2.2 Mixed-Stream Batch Packing and Attention

Encoded examples of both streams are stacked along the batch dimension,

$$X^{\text{pack}} = \text{Concat}_B (X^{\text{manip}}, X^{\text{nav}}) \in \mathbb{R}^{(B_{\text{manip}} + B_{\text{nav}}) \times L \times D}, \quad (7)$$

where B_{manip} and B_{nav} need not be equal. Visual tokens and conditioning are packed in the same order, and attention never crosses batch rows. A navigation row therefore conditions only on navigation views and actions, and a manipulation row only on manipulation ones, while both update the same DiT parameters. The two streams therefore interact only through the weights they share.

Within a row, the video DiT reads visual tokens, while language and robot state enter through cross-attention. Current-observation tokens cannot read future frames, future-frame tokens may read the full visual sequence, and action tokens cannot read future video tokens. Let $\mathcal{V}_t$ denote the current visual tokens and $\mathcal{T}^s$ the learnable tokens of stream s . The action token $a_{t,j}^s$ attends to

$$\mathcal{N}(a_{t,j}^s) = \mathcal{V}_t \cup \mathcal{T}^s \cup \{a_{t,k}^s : k \leq j\}, \quad (8)$$

so each action reads the current visual context, the learnable tokens, itself, and earlier actions, but not later actions. Learnable tokens read the current visual context and one another, but not action tokens. Under prefix-conditioned asynchronous inference, the first P actions of a chunk are the clean actions already committed for execution. The causal mask of Eq. 8 lets the later actions condition on this clean prefix without leaking information from later noisy actions backward, so each new chunk continues the executed motion consistently. At deployment, both streams run continuously in one batch without switching model parameters or invoking a separate policy.

3.2.3 Independent Stream Sampling

In a complete mobile-manipulation trajectory, the arm is mostly idle during navigation and the base is mostly idle during manipulation. We therefore split each mobile-manipulation trajectory by rule into navigation windows and manipulation windows, one set for each stream. Within each stream, a second rule marks every window as active or idle, depending on whether the controlled part of the robot moves. Each window then becomes a separate example of its stream. A mobile-manipulation trajectory thus contributes to both streams, whereas navigation-only data such as MAP-Data and manipulation-only data contribute only to their own stream. Examples are drawn from

$$q(d, i) = \frac{\alpha_d}{\sum_{d'} \alpha_{d'}} \frac{w_{d,i}}{\sum_j w_{d,j}}, \quad (9)$$

where α_d is the weight of source d and $w_{d,i}$ the weight of its i -th window. Normalizing within each source decouples its sampling mass from its size, and the window weights set the proportions of navigation and manipulation windows and of active and idle windows. In this way, idle windows are retained at a controlled proportion instead of dominating training. Duplicate or temporally adjacent windows are rejected within a batch, and windows from the same episode must have start frames separated by at least δ frames. The selected examples are grouped by stream and packed with Eq. 7. The stream-specific batch sizes may differ, and navigation and manipulation rows need not originate from the same trajectory.

3.2.4 Image-Plane Auxiliary Supervision

Both streams predict spatial actions from images. To ground these actions in the image, we append image-plane targets to each action vector,

$$a_{t,i}^s = [a_{t,i}^{s,\text{act}}, a_{t,i}^{s,\text{aux}}], \quad (10)$$

and denoise them jointly with the actions, as shown in the output row of Fig. 3. For navigation, the targets are the target bounding box and the projected MAP position of Eq. 3, which indicate where the target is and where the base should stop at every step of the approach. For manipulation, the targets are the two end-effector trajectories projected into the main camera view, together with visibility flags. Predicting these tracks jointly with the camera-frame end-effector poses explicitly links the camera-frame actions to their image-plane locations in the main view. The projected MAP and

Algorithm 1 Calibrated main-view end-effector geometry

Require: states $S_{0:T-1}$; main-view video V ; intrinsics K ; reference-to-camera transform $E_{c \leftarrow \text{ref}}$; per-arm base-to-reference transforms $\{M_{\text{ref} \leftarrow b_i}^i\}_{i \in \mathcal{A}}$; operation mask m^{op} ; point offsets $\mathbf{o}_{J6} = \mathbf{0}$ and $\mathbf{o}_{\text{vis}}$; arms $\mathcal{A} = \{\text{left}, \text{right}\}$

Ensure: flange and gripper image points $P_{0:T-1}^{J6}$ and $P_{0:T-1}^{\text{vis}}$; geometric validity mask $m_{0:T-1}^{\text{geo}}$; provenance record $\mathcal{R}$

Notation: $h(\mathbf{y}) = [\mathbf{y}^\top, 1]^\top$; $\text{xyz}(\cdot)$ drops the homogeneous coordinate; $\pi_K([X, Y, Z]^\top) = [f_x X/Z + c_x, f_y Y/Z + c_y]^\top$

```
1:  $\mathcal{R} \leftarrow \text{VALIDATECALIBRATION}(K, E_{c \leftarrow \text{ref}}, \{M_{\text{ref} \leftarrow b_i}^i\})$ 
2: for  $t = 0, \dots, T-1$  and  $i \in \mathcal{A}$  do
3:    $(\mathbf{x}_t^i, R_t^i) \leftarrow \text{DECODESTATE}(S_t, i)$  ▷ pose in the arm base frame
4:   for  $\rho \in \{J6, \text{vis}\}$  do
5:      $\mathbf{c}_t^{i,\rho} \leftarrow \text{xyz}(E_{c \leftarrow \text{ref}} M_{\text{ref} \leftarrow b_i}^i h(\mathbf{x}_t^i + R_t^i \mathbf{o}_\rho))$  ▷ point in the camera frame
6:      $P_t^{i,\rho} \leftarrow \pi_K(\mathbf{c}_t^{i,\rho})$ 
7:   end for
8:    $m_t^{\text{geo},i} \leftarrow \mathbf{1}[(\mathbf{c}_t^{i,\text{vis}})_z > 0] \cdot \mathbf{1}[P_t^{i,\text{vis}} \in [0, W_{\text{img}}) \times [0, H_{\text{img}})] \cdot m_t^{\text{op}}$ 
9: end for
10:  $\mathcal{R} \leftarrow \mathcal{R} \cup \text{HASHINPUTS}(S, V, K, E_{c \leftarrow \text{ref}}, \{M_{\text{ref} \leftarrow b_i}^i\})$ 
11: return  $P^{J6}, P^{\text{vis}}, m^{\text{geo}}, \mathcal{R}$ 
```

end-effector positions are image-plane counterparts of the navigation goal and the manipulation action, which keeps the auxiliary targets geometrically consistent with the actions they accompany.

The end-effector tracks are generated from recorded robot states and camera calibration in three steps. Algorithm 1 projects each end effector into the main view, Algorithm 2 decides in which frames the projected point is visible, and Algorithm 3 runs both steps on every episode and validates the output.

Calibrated geometry. For each arm, Algorithm 1 decodes the end-effector pose in the native base frame of that arm, maps it into a common reference frame with the per-arm transform $M_{\text{ref} \leftarrow b_i}^i$, and then into the main camera frame with $E_{c \leftarrow \text{ref}}$. Two points are tracked per arm, namely the flange of the sixth joint with zero offset and a point on the gripper with offset $\mathbf{o}_{\text{vis}}$, which is the part seen in the image. Both points are projected with the pinhole model π_K . A projected gripper point is geometrically valid if it lies in front of the camera, falls inside the image, and belongs to the manipulation interval given by the operation mask m^{op} . The calibration is checked before projection, and a record of its inputs is stored with the labels, so every label can be traced to its calibration.

Appearance-gated visibility. A geometrically valid point may still be hidden, for example by the other arm or by an object. Algorithm 2 thus checks the image content at the projected point. It encodes every k -th frame with DINOv2 [21], takes the patch token at the projected gripper point, and scores it by its highest cosine similarity to positive end-effector prototypes minus its highest similarity to negative background prototypes. The scores are interpolated only within contiguous valid segments, thresholded with hysteresis so that the visibility does not flicker near the threshold, and cleaned by filling short gaps and removing short runs. The output keeps the coordinates only in visible frames and labels each frame as visible, valid but occluded, or invalid for the auxiliary loss.

Label generation. Algorithm 3 runs the two steps on every episode in parallel. It resolves the calibration of each episode from a registry, writes the labels together with a per-episode report, and validates all outputs against the dataset before training. Coordinates are normalized to the annotation canvas, and only visible points enter the auxiliary loss.

3.2.5 Training Objective

All targets are trained with flow matching [19]. For a clean video-latent sequence or normalized action chunk $\mathbf{x}$, noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, and noise level $\sigma \in [0, 1]$, the noisy input and the velocity target are

$$\mathbf{x}_\sigma = (1 - \sigma)\mathbf{x} + \sigma\epsilon, \quad \mathbf{v}^* = \epsilon - \mathbf{x}. \quad (11)$$

Video and action noise levels are sampled independently for each example as $\sigma = \kappa u / [1 + (\kappa - 1)u]$ with $u \sim \mathcal{U}(0, 1)$ and $\kappa = 5$. The current observation remains clean, and only future video latents are supervised. The action and auxiliary dimensions of one action vector share a noise level. For prefix-conditioned training, positions $1 \leq t \leq P$ are

Algorithm 2 Appearance-gated visibility of the end-effector track

Require: main-view video V ; gripper points P^{vis} ; geometric validity mask m^{geo} ; DINOv2 encoder Φ ; positive and negative prototypes Q^+, Q^- ; stride k ; thresholds $\eta_{\text{in}} > \eta_{\text{out}}$; minimum gap and run lengths $n_{\text{gap}}, n_{\text{run}}$; arms $\mathcal{A}$

Ensure: visibility mask $z_{0:T-1}$; visible-only coordinates $\tilde{P}_{0:T-1}$; quality labels $q_{0:T-1}$

```
1:  $U^+ \leftarrow \text{ENCODEPROTOTYPES}(\Phi, Q^+)$ ,  $U^- \leftarrow \text{ENCODEPROTOTYPES}(\Phi, Q^-)$ 
2: for  $t \in \{0, k, 2k, \dots\}$  with  $t < T$  do
3:    $F_t \leftarrow \Phi(V_t)$ 
4:   for  $i \in \mathcal{A}$  do
5:     if  $m_t^{\text{geo}, i} = 1$  then
6:        $\mathbf{f} \leftarrow \text{PATCHTOKEN}(F_t, P_t^{i, \text{vis}})$  ▷ token at the projected point
7:        $\mu_t^i \leftarrow \max_{\mathbf{u} \in U^+} \cos(\mathbf{f}, \mathbf{u}) - \max_{\mathbf{u} \in U^-} \cos(\mathbf{f}, \mathbf{u})$ 
8:     else
9:        $\mu_t^i \leftarrow \text{NaN}$ 
10:    end if
11:  end for
12: end for
13:  $\bar{\mu} \leftarrow \text{INTERPOLATEWITHINSEGMENTS}(\mu, m^{\text{geo}})$ 
14:  $z \leftarrow \text{HYSTERESIS}(\bar{\mu}, \eta_{\text{in}}, \eta_{\text{out}}) \cdot m^{\text{geo}}$ 
15:  $z \leftarrow \text{MORPHOLOGY}(z, m^{\text{geo}}, n_{\text{gap}}, n_{\text{run}})$  ▷ fill short gaps, remove short runs
16: for  $t = 0, \dots, T-1$  and  $i \in \mathcal{A}$  do
17:    $\tilde{P}_t^i \leftarrow P_t^{i, \text{vis}}$  if  $z_t^i = 1$ , else NaN
18:    $q_t^i \leftarrow 1$  if  $z_t^i = 1$ ; 2 if  $m_t^{\text{geo}, i} = 1$  and  $z_t^i = 0$ ; 0 otherwise ▷ visible, occluded, invalid
19: end for
20: return  $z, \tilde{P}, q$ 
```

Algorithm 3 End-effector label generation for a dataset

Require: episodes $\mathcal{D}$; calibration registry $\mathcal{C}$; point offsets $\mathbf{o}_{\text{J6}}, \mathbf{o}_{\text{vis}}$; visibility parameters $\Theta_{\text{vis}} = (\Phi, Q^+, Q^-, k, \eta_{\text{in}}, \eta_{\text{out}}, n_{\text{gap}}, n_{\text{run}})$

Ensure: per-episode labels and reports; validated manifest $\mathcal{M}$

```
1: for all episodes  $e \in \mathcal{D}$  in parallel do
2:    $(S^e, V^e) \leftarrow \text{READEPISODE}(e)$ 
3:    $(K^e, E^e, \{M^{i, e}\}, m^{\text{op}, e}) \leftarrow \text{RESOLVECALIBRATION}(e, \mathcal{C})$ 
4:    $(P^{\text{J6}, e}, P^{\text{vis}, e}, m^{\text{geo}, e}, \mathcal{R}^e) \leftarrow \text{Algorithm 1}$ 
5:    $(z^e, \tilde{P}^e, q^e) \leftarrow \text{Algorithm 2 with } \Theta_{\text{vis}}$ 
6:    $\text{WRITELABELS}(e, P^{\text{J6}, e}, P^{\text{vis}, e}, \tilde{P}^e, m^{\text{geo}, e}, z^e, q^e, \mathcal{R}^e)$  ▷ labels and report
7: end for
8:  $\mathcal{M} \leftarrow \text{VALIDATEALL}(\mathcal{D})$  ▷ every episode has complete labels
9: return  $\mathcal{M}$ 
```

replaced by clean committed actions and excluded from supervision. With probability p_{pre} , P is drawn uniformly from $\{P_{\text{min}}, \dots, P_{\text{max}}\}$, and otherwise $P = 0$, which matches the prefix lengths used at deployment.

For each supervised group g , let $\mathcal{F}_{b, t}^g$ contain the valid features at position t of example b , and let $\mathcal{I}_b^g$ contain the non-prefix, non-padded positions with at least one valid feature. The per-example loss is

$$\ell_b^g = \frac{\omega(\sigma_b)}{|\mathcal{I}_b^g|} \sum_{t \in \mathcal{I}_b^g} \frac{1}{|\mathcal{F}_{b, t}^g|} \sum_{j \in \mathcal{F}_{b, t}^g} \left(\hat{v}_{b, t, j} - v_{b, t, j}^* \right)^2, \quad (12)$$

where $\omega(\sigma_b)$ is a loss weight that depends on the noise level. Each group loss $\mathcal{L}^g$ is the mean of ℓ_b^g over the examples with valid supervision, using counts aggregated across workers. The full objective is

$$\mathcal{L} = \lambda_{\text{vid}} \mathcal{L}_{\text{vid}} + \sum_{s \in \{\text{nav}, \text{manip}\}} (\lambda_{\text{act}}^s \mathcal{L}_{\text{act}}^s + \lambda_{\text{aux}}^s \mathcal{L}_{\text{aux}}^s), \quad (13)$$

where $\mathcal{L}_{\text{vid}}$ supervises future video latents of both streams, $\mathcal{L}_{\text{act}}^s$ the actions, and $\mathcal{L}_{\text{aux}}^s$ the image-plane targets. Because each term is averaged only over valid positions and features, a sample contributes only to the losses of its own stream.

3.2.6 Implementation and Inference

Network and optimization. UniWAM uses the Wan2.2-TI2V-5B [27] video backbone, video VAE, and text conditioning. The action DiT has 30 transformer layers and a hidden dimension of $D_a = 1024$, compared with $D_v = 3072$ for the video DiT. Sampling every fourth image from a 33-frame observation window yields nine RGB frames, which are encoded into three temporal VAE latents. Camera views are arranged into a 384×320 canvas with a main view and wrist panels. We optimize with AdamW, an initial learning rate of 10^{-5} , and cosine decay. The video VAE and the text encoder stay frozen, and all other parameters are trained. Inference uses a deterministic schedule with ten Euler denoising steps and no classifier-free guidance for either stream.

Infrastructure acceleration. In standard PyTorch execution, each transformer operation is dispatched separately from Python. Because the mixed video-action layers operate on relatively short action sequences, the CPU may spend substantial time submitting kernels while the GPU waits for the next launch. This overhead affects every training step and becomes more pronounced during inference, where the action denoising computation is repeated over multiple Euler steps. UniWAM reduces this overhead through separate training and inference paths. During training, `torch.compile` captures the tensor-only mixed-stream transformer layer as a full graph and reuses the compiled function across layers and optimization steps, while data preparation, loss computation, and parameter updates remain outside the compiled region. During inference, we compile the VAE encoder, the visual-cache prefill, and the cached action-denoising function separately. The current observation is encoded once, and its layerwise video keys and values are reused across the ten Euler steps, so the video branch is not recomputed at every step. For fixed input shapes, reduce-overhead compilation and execution compatible with CUDA graphs further reduce the cost of CPU submission and kernel launches. When both streams are active, navigation and manipulation rows are packed into the same batch and processed by the same compiled functions, which increases the work in each launch and avoids separate model invocations. In our measurements, this implementation reduces two-stream inference latency on one NVIDIA H200 from about 220 ms to below 60 ms, and the corresponding training setup exceeds 600 samples per second on 16 NVIDIA H200 GPUs. Asynchronous execution further overlaps the remaining inference time with the execution of committed actions.

4 Experiments

We evaluate UniWAM on MAP-Bench and on 24 real-robot tasks. The experiments answer four questions. First, does UniWAM dock more accurately than existing policies, and how much does MAP-Data help? Second, does this advantage carry over to real robots and to complete mobile-manipulation tasks? Third, which part of the gain comes from the mixed-stream architecture and which from independent sampling? Fourth, how does each design component contribute?

4.1 Experimental Setup

Tasks and platforms. MAP-Bench evaluates MAP navigation from held-out initial poses in the three distance ranges of Sec. 3.1. The real-robot evaluation covers 13 MAP Navigation, 4 Mobile Manipulation, and 7 Manipulation tasks on four embodiments, namely AgileX Cobot Magic, AgiBot G2, dual-arm Franka Research 3, and an AgileX single-arm platform. The 13 MAP Navigation tasks run on both the Cobot Magic and the AgiBot G2. The four Mobile Manipulation tasks and Manipulation task 5 run on the Cobot Magic, Manipulation tasks 1 to 3 and 7 on the dual Franka, and Manipulation tasks 4 and 6 on the AgileX single arm, so every embodiment contributes to the evaluation.

Fig. 4 shows the mobile platform, whose two head cameras realize the stream-specific observations of UniWAM. The wide-view camera captures distant targets and free space for navigation, and the downward-facing camera captures the workspace for manipulation. The navigation stream observes the wide-view head camera, and the manipulation stream observes the downward-facing head camera and the two wrist cameras. In Mobile Manipulation, the navigation stream also observes the wrist cameras to track the arms during the approach.

UniWAM variants. Both variants start from Wan2.2-TI2V-5B [27] without embodied pretraining. **UniWAM-Base** uses only the data available to the baselines and is the matched comparison. It is trained on the 35K-trajectory training split on MAP-Bench and on the demonstrations of the 24 evaluation tasks on real robots. **UniWAM-Scaled** keeps the architecture and adds data through independent stream sampling, drawing large-scale and task data at a 7:3 ratio. On



Figure 4 Mobile manipulation platform. A shallow-angle head camera with a wide field of view provides the navigation-stream input, and a steep-angle head camera with a narrow field of view, together with the wrist cameras, provides the manipulation-stream input. In Mobile Manipulation, the navigation stream also receives the wrist cameras. The two inputs form the navigation and manipulation rows of a mixed-stream batch, which UniWAM processes in one forward pass.

MAP-Bench, it adds MAP-Data under the same training budget, so its gain isolates the value of MAP-Data. On real robots, it adds MAP-Data to the navigation stream, sampling 15 trajectories per navigation target, and adds 402 private manipulation tasks with 30 demonstrations each to the manipulation stream. These tasks are disjoint from the evaluation tasks. In both variants, windows of mobile-manipulation trajectories in which the arm is idle during base motion are sampled at 20% of their original probability, and windows in which the base is idle during manipulation at 15%. Each variant is a single model for all tasks and embodiments, without task-specific fine-tuning.

MAP-Bench baselines. We compare with the VLA models $\pi_{0.5}$ [3] and GR00T N1.7 [2], the VLN models UniNaVid [42] and InternVLA-N1 [15], and the world-action model FastWAM [41]. All models, including UniWAM, are trained on the MAP-Bench training split for 40,000 steps on 16 NVIDIA H200 GPUs. All models observe only the forward RGB camera, and each keeps its native action interface, into which the navigation labels are converted. Only UniWAM receives the bounding-box and MAP auxiliary targets. $\pi_{0.5}$ and GR00T N1.7 are fine-tuned with all parameters from their public checkpoints. InternVLA-N1 starts from the released DualVLN checkpoint [29] and, as in its official recipe, keeps System 2 frozen and trains only the System 1 trajectory generator, using the current frame and up to four history frames. UniNaVid is fine-tuned with all parameters from the UniNaVid-7B checkpoint, using the current observation and up to three earlier observations sampled every 0.5 s. Because its native outputs are discrete forward and turning primitives, its labels are the primitive sequences that best fit the future trajectory, found by dynamic programming. FastWAM is the baseline closest to UniWAM in formulation. It starts from Wan2.2-TI2V-5B and shares the data, visual inputs, action targets, horizon, optimizer, and schedule of UniWAM-Base, but follows its own official architecture and does not use the stream-specific interfaces, the causal action mask, or the image-plane auxiliary supervision. Apart from the shared data and budget, each baseline follows its official fine-tuning recipe.

Real-robot baselines. We compare with $\pi_{0.5}$ and GR00T N1.7, which start from their public checkpoints, keep their native action interfaces, and update all parameters. All methods are trained on the same demonstrations of the 24 evaluation tasks with the same instructions. Each MAP Navigation task has 50 demonstrations on each of its two embodiments, and each Mobile Manipulation or Manipulation task has about 200, with 400 for selected challenging tasks such as six-cup stacking. All methods observe the same cameras. In MAP Navigation, every method observes only the wide-view navigation head camera. In Mobile Manipulation, all methods observe the navigation head camera, the manipulation head camera, and the two wrist cameras. UniWAM assigns them to its streams, whereas $\pi_{0.5}$ and GR00T N1.7 receive all four views as a single multi-view observation. For this four-camera setting, GR00T N1.7 uses a bimanual embodiment configuration similar to that of ALOHA, with one video key per camera. For MAP Navigation,

Table 3 Real-robot training settings. Steps and GPUs are given per model. Each baseline trains one MAP Navigation model per mobile robot and one model for the remaining tasks of each embodiment, whereas each UniWAM variant is one model for all tasks.

	$\pi_{0.5}$	GR00T N1.7	UniWAM-Base	UniWAM-Scaled
Initialization	$\pi_{0.5}$ base checkpoint	GR00T N1.7-3B checkpoint	Wan2.2-TI2V-5B	Wan2.2-TI2V-5B
Robot pretraining	yes	yes	none	none
Frozen modules	none	none	VAE, text encoder	VAE, text encoder
Task demonstrations	24 tasks	24 tasks	24 tasks	24 tasks
Additional data	none	none	none	MAP-Data and 402 tasks
Image-plane auxiliary outputs	✗	✗	✓	✓
Steps per model	50K	50K	20K	30K
GPUs per model	8	8	32	64

each baseline is trained as one model on all 13 tasks for each of the Cobot Magic and the AgiBot G2, because these tasks share the action space and differ mainly in the target. For the other tasks, each baseline is fine-tuned separately for each embodiment. We also co-trained each baseline on all 24 tasks, as for UniWAM, but this performed worse than per-embodiment training for both $\pi_{0.5}$ and GR00T N1.7, which we attribute to their large-scale robot pretraining. We therefore report their per-embodiment models, each evaluated at the checkpoint that performed best in our real-robot tests. Table 3 summarizes the training settings of all real-robot models, including initialization, data, and compute. Sharing the data, cameras, and evaluation protocol across all methods keeps the comparison fair.

Real-robot evaluation protocol. The reference MAP of each task is the parking pose marked during data collection. In each MAP Navigation trial, the robot starts at this pose and is moved to a random initial pose 1 to 5 m away with a heading offset within $\pm 45^\circ$. The onboard state estimator records both motions, so the reached pose is scored against the same reference. The robot runs until it stops under the rule of Sec. 3.1. Each Mobile Manipulation trial consists of navigation followed by manipulation. When the base stops, E_{pos} and E_{head} are computed against the reference MAP, and the robot then executes the manipulation stage. At test time, UniWAM activates both streams and packs them into one batch throughout each trial. The policies still predict small but nonzero base motions during manipulation, which can disturb precise placements. Therefore, for all methods, a predicted base chunk is not executed when its displacement stays below 0.05 m and 0.05 rad, analogous to the stopping rule of MAP-Bench. Each Manipulation trial starts from a fixed working pose, and a task-specific predicate decides success. Each task is evaluated over 40 trials on each embodiment it runs on, and MAP Navigation results are averaged over the Cobot Magic and the AgiBot G2. The initial configurations of these trials, including the robot pose and the object placements, are recorded once and reproduced for every method. All methods share task definitions, instructions, success criteria, and termination rules.

Metrics. MAP Navigation is measured by E_{pos} and E_{head} of Eq. 5. On real robots, the reached pose is obtained from the onboard state estimator. Mobile Manipulation reports the same errors when the base stops, together with the success rate (SR) of the subsequent manipulation. Manipulation reports SR. Real-robot results are unweighted task means.

4.2 MAP-Bench Results

Table 4 summarizes MAP-Bench performance across models and initial distances.

Overall comparison. UniWAM-Scaled achieves the lowest errors in every distance range, reaching 9.85 cm and 2.30° overall. These errors are 30.1% and 44.0% lower than those of the strongest external baselines, GR00T N1.7 at 14.1 cm and $\pi_{0.5}$ at 4.11° , respectively. Without MAP-Data, UniWAM-Base already reduces these errors by 11.3% and 21.2%, which shows that the architecture alone improves docking under identical data.

Semantic versus manipulation-ready navigation. Although trained on the same split, the VLN models retain large terminal errors, with Overall position errors of 48.2 and 62.3 cm and long-range errors of 87.2 and 130.0 cm. Both are designed for semantic navigation, and their official recipes keep the planning module of InternVLA-N1 frozen and restrict Uni-NaVid to discrete motion primitives. The VLA and WAM baselines are more accurate but degrade with distance. For example, the position error of $\pi_{0.5}$ increases from 7.98 cm at short range to 50.8 cm at long range.

Long-range docking. Relative to the best baseline in each range, UniWAM-Scaled reduces position error by 6.3% at short range and 35.5% at long range. Its position error grows by a factor of 1.9 from short to long range, compared with

Table 4 MAP-Bench results over the evaluation split. Bold and underline mark the best and second-best results.

Model	E_{pos} (cm) ↓			Overall		E_{head} (°) ↓			Overall
	Short	Mid	Long			Short	Mid	Long	
Vision-language-action models (VLA)									
GR00T N1.7 [2]	9.48	10.8	22.0	14.1		5.66	5.42	7.46	6.18
$\pi_{0.5}$ [3]	7.98	9.85	50.8	22.9		3.26	3.51	5.55	4.11
Vision-language navigation models (VLN)									
Uni-NaVid [42]	17.3	40.0	87.2	48.2		7.98	15.0	23.4	15.5
InternVLA-N1 [15]	17.3	39.7	130.0	62.3		3.72	11.0	27.6	14.1
World-action models (WAM)									
FastWAM [41]	9.24	11.0	31.1	17.1		3.44	3.95	6.66	4.68
UniWAM-Base	<u>7.71</u>	<u>8.85</u>	<u>21.0</u>	<u>12.5</u>		<u>2.37</u>	<u>3.06</u>	<u>4.29</u>	<u>3.24</u>
UniWAM-Scaled	7.48	7.86	14.2	9.85		1.82	2.08	3.01	2.30

Table 5 MAP-Bench errors when training with a fraction of the MAP-Data targets under a fixed compute budget. The second column is the fraction of MAP-Data targets from which the MAP-Data samples are drawn. Whenever MAP-Data is used, it is mixed with the MAP-Bench training split at a fixed 7:3 ratio, so all runs with MAP-Data see the same number of MAP-Data samples. 0% denotes training on the MAP-Bench split alone. UniWAM at 0% and 100% are UniWAM-Base and UniWAM-Scaled, and $\pi_{0.5}$ uses the same data and budget. Position errors are in centimeters and heading errors in degrees.

Model	MAP-Data targets	E_{pos} Short	E_{pos} Mid	E_{pos} Long	E_{pos} Overall	E_{head} Overall
UniWAM-Base	0%	7.71	8.85	21.0	12.5	3.24
UniWAM	25%	7.63	8.52	18.1	11.4	2.81
UniWAM	50%	7.52	8.03	15.3	10.3	2.43
UniWAM-Scaled	100%	7.48	7.86	14.2	9.85	2.30
$\pi_{0.5}$	0%	7.98	9.85	50.8	22.9	4.11
$\pi_{0.5}$	100%	7.42	9.45	47.8	21.6	4.07

2.3 for GR00T N1.7, 3.4 for FastWAM, and 6.4 for $\pi_{0.5}$. Its heading error is also 40.7% to 45.8% lower than that of the best baseline in each of the three distance ranges.

Effect of MAP-Data. UniWAM-Base and UniWAM-Scaled share the architecture and training budget, and only UniWAM-Scaled mixes MAP-Data. MAP-Data reduces the Overall position and heading errors by 21.2% and 29.0%, and the position gain increases from 3.0% at short range to 32.4% at long range. Long approaches thus benefit most from explicit supervision of where to stop, since small heading errors grow over distance.

Scaling with MAP-Data. This experiment asks whether navigation keeps improving as more MAP-Data becomes available. We train UniWAM on MAP-Bench with a random subset of 25%, 50%, or 100% of the MAP-Data targets, sampling 15 trajectories per target and excluding every benchmark target. All other settings follow UniWAM-Scaled, including the architecture, the 7:3 mixture of MAP-Data and the MAP-Bench training split, and 40,000 steps on 16 GPUs. Because the mixture ratio and the number of steps are fixed, the 25%, 50%, and 100% runs see the same number of MAP-Data samples and differ only in how many distinct targets and scenes these samples come from. Table 5 shows that both errors decrease monotonically as more of MAP-Data is used. The gain concentrates at long range, where the position error falls from 21.0 to 14.2 cm, whereas the short-range error changes by only 0.23 cm. The gain diminishes with more data but remains positive, and the last doubling still reduces the Overall position error by 4.4%.

To test whether other models benefit equally, we also train $\pi_{0.5}$ with the full MAP-Data under the same mixture and budget. Its Overall errors fall by only 5.7% and 1.0%, compared with 21.2% and 29.0% for UniWAM, and its long-range position error remains at 47.8 cm, compared with 14.2 cm for UniWAM-Scaled. $\pi_{0.5}$ gains mainly at short range, where it reaches 7.42 cm, on par with UniWAM-Scaled. Large-scale MAP supervision thus helps most when navigation is learned as its own stream with image-plane targets, as in UniWAM.

Table 6 Real-robot results averaged over the tasks in each category. Bold and underline mark the best and second-best results.

Model	MAP Navigation		Mobile Manipulation			Manipulation
	E_{pos} (cm) ↓	E_{head} (°) ↓	E_{pos} (cm) ↓	E_{head} (°) ↓	SR (%) ↑	SR (%) ↑
$\pi_{0.5}$ [3]	23.3	5.15	26.2	5.72	62.5	71.4
GR00T N1.7 [2]	18.1	6.55	20.4	7.20	39.4	36.8
UniWAM-Base	<u>15.1</u>	<u>4.91</u>	<u>17.1</u>	<u>5.30</u>	<u>63.1</u>	44.3
UniWAM-Scaled	11.7	4.25	13.2	4.56	80.0	<u>68.9</u>

4.3 Real-Robot Results

Table 6 summarizes the 24 real-robot tasks, and Fig. 6 in Sec. 4.6 shows representative executions. Among the evaluated models, UniWAM-Scaled achieves the lowest position and heading errors in MAP Navigation and Mobile Manipulation, the highest Mobile Manipulation success rate, and the second-highest Manipulation success rate.

MAP navigation. The two baselines trade off position and heading accuracy. GR00T N1.7 has the lower position error, 18.1 cm compared with 23.3 cm for $\pi_{0.5}$, whereas $\pi_{0.5}$ has the lower heading error, 5.15° compared with 6.55°. Trained only on task data, UniWAM-Base outperforms both on both metrics, reaching 15.1 cm and 4.91°. UniWAM-Scaled further reduces the errors to 11.7 cm and 4.25°, 35.4% and 17.5% below the strongest baseline, respectively. It achieves the lowest position error on all 13 tasks and the lowest heading error on 11.

Mobile manipulation. The docking accuracy of UniWAM-Scaled carries over to end-to-end performance. It stops 13.2 cm and 4.56° from the MAP, 35.3% and 20.3% below the best baseline, respectively. Its success rate reaches 80.0%, 17.5 percentage points above $\pi_{0.5}$ at 62.5%, with non-overlapping 95% intervals, and it is highest on all four tasks. UniWAM-Base reaches a success rate similar to that of $\pi_{0.5}$, 63.1% versus 62.5%, while docking more accurately.

Manipulation. $\pi_{0.5}$ achieves the highest average success at 71.4%, followed by UniWAM-Scaled at 68.9%. UniWAM-Scaled leads on the three sorting and placement tasks, while $\pi_{0.5}$ leads by 2.5 to 12.5 percentage points on six-cup stacking, the two tube-insertion tasks, and long-horizon coffee preparation. $\pi_{0.5}$ is pretrained on large-scale robot data, whereas UniWAM starts from a video model and relies on mixed-data training, so the remaining gap is largest on these dexterous or long-horizon tasks. GR00T N1.7 has the lowest success at 36.8%.

Scaling. Relative to UniWAM-Base, UniWAM-Scaled reduces MAP Navigation errors by 22.5% and 13.4% and raises success by 16.9 and 24.6 percentage points on Mobile Manipulation and Manipulation. These gains reflect both more data and more compute, and Sec. 4.2 isolates the effect of MAP-Data.

Per-task results. Table 7 reports the per-task results. In MAP Navigation, UniWAM-Scaled has the lowest position error on all 13 tasks and the lowest heading error on 11. On the sink and the round side table, UniWAM-Base has the lowest heading error. In Mobile Manipulation, UniWAM-Scaled is best on all four tasks in all three metrics. UniWAM-Base exceeds $\pi_{0.5}$ in success on cube sorting and pouring and trails it by 2.5 points on the other two. In Manipulation, UniWAM-Scaled leads on tasks 4 to 6, and $\pi_{0.5}$ leads on tasks 1 to 3 and 7, with the largest gap of 12.5 points on coffee preparation, where GR00T N1.7 fails every trial and $\pi_{0.5}$ reaches 52.5%.

Zero-shot transfer from MAP-Data to real robots. This experiment tests whether MAP-Data alone teaches real-robot docking. Each model is trained only on MAP-Data for 70,000 steps on 32 NVIDIA H200 GPUs and is deployed directly on the Cobot Magic and the AgiBot G2. We refer to UniWAM trained in this way as UniWAM-Sim. The baselines use the same data and budget and otherwise follow their official recipes. Evaluation uses the 13 real MAP Navigation tasks and the protocol above, with 40 trials per task on each embodiment, and the errors are averaged over the two embodiments as for the real-data models. No real-robot data, room, or target instance is used in training, so every real-robot test is unseen. Table 8 shows that UniWAM-Sim stops 18.7 cm and 6.34° from the MAP on average. Without any real-robot data, its position error is lower than that of $\pi_{0.5}$ trained on the real demonstrations, although its heading error remains higher than the 5.15° of that model. The baselines transfer less well. The position errors of $\pi_{0.5}$ and GR00T N1.7 are 2.4 and 1.9 times those of their real-data versions in Table 6, and FastWAM, which shares the video backbone of UniWAM, remains 12.1 cm behind UniWAM-Sim. This shows that navigation learned from MAP-Data transfers to real robots, and the gap to FastWAM is consistent with the stream-specific design and image-plane supervision.

Table 7 Per-task real-robot results with 40 trials per task on each embodiment. MAP Navigation cells report position error and heading error averaged over the Cobot Magic and the AgiBot G2. Mobile Manipulation cells additionally report the success rate in percent, and Manipulation cells report only the success rate. Avg. is the unweighted mean over the tasks in each category. Bold and underline mark the best and second-best values for each metric.

ID Task	$\pi_{0.5}$	GR00T N1.7	UniWAM-Base	UniWAM-Scaled
MAP Navigation E_{pos} (cm) ↓ / E_{head} (°) ↓				
1 Water dispenser	18.9 / 4.83	19.7 / 5.47	<u>13.9</u> / <u>4.23</u>	10.3 / 3.86
2 Humanoid robot	24.7 / 5.17	18.4 / 6.57	<u>15.3</u> / <u>4.84</u>	12.6 / 4.47
3 TV cabinet	20.2 / 4.56	14.8 / 5.74	<u>12.1</u> / <u>3.76</u>	8.94 / 3.58
4 Tool cabinet	25.3 / <u>5.63</u>	20.2 / 7.18	<u>16.5</u> / 7.27	11.7 / 4.06
5 Cushion	21.8 / 4.69	15.6 / 5.93	<u>13.2</u> / <u>4.34</u>	9.73 / 3.74
6 Table	28.1 / 5.84	19.7 / 7.46	<u>16.9</u> / <u>5.47</u>	13.7 / 4.86
7 Sink	16.9 / 4.27	17.3 / 6.34	<u>12.7</u> / 3.86	10.8 / <u>4.13</u>
8 Potted plant	30.9 / <u>6.36</u>	23.1 / 8.13	<u>19.7</u> / 6.58	14.8 / 5.24
9 Wall-side cabinet	23.7 / 5.14	18.2 / 6.77	<u>14.8</u> / <u>4.79</u>	11.9 / 4.31
10 Workstation	24.9 / 5.46	18.8 / 6.86	<u>15.9</u> / <u>5.24</u>	13.1 / 4.72
11 Black side cabinet	26.4 / <u>5.91</u>	21.1 / 7.57	<u>17.1</u> / 5.96	12.8 / 4.57
12 Round side table	21.2 / 4.76	15.1 / 5.84	<u>14.5</u> / 3.94	11.8 / <u>4.24</u>
13 Countertop	19.5 / 4.35	13.9 / 5.24	<u>13.4</u> / <u>3.53</u>	9.41 / 3.46
Avg.	23.3 / 5.15	18.1 / 6.55	<u>15.1</u> / <u>4.91</u>	11.7 / 4.25
Mobile Manipulation E_{pos} (cm) ↓ / E_{head} (°) ↓ / SR (%) ↑				
1 Place items in storage box	25.6/5.67/65.0	19.2/6.91/42.5	<u>16.7/5.16/62.5</u>	12.8/4.37/82.5
2 Sort colored cubes into trays	23.8/ <u>5.29</u> /55.0	21.3/7.46/35.0	<u>18.2/5.73/60.0</u>	13.1/4.46/77.5
3 Clear cups onto tray	29.1/6.18/ <u>70.0</u>	18.7/6.58/47.5	<u>15.9/4.87/67.5</u>	13.8/4.83/85.0
4 Pour water into cup	26.3/5.74/60.0	22.4/7.83/32.5	<u>17.6/5.42/62.5</u>	13.1/4.57/75.0
Avg.	26.2/5.72/62.5	20.4/7.20/39.4	<u>17.1/5.30/63.1</u>	13.2/4.56/80.0
Manipulation SR (%) ↑				
1 Stack six-cup pyramid	75.0	35.0	45.0	<u>72.5</u>
2 Insert tubes into metal rack	77.5	35.0	42.5	<u>67.5</u>
3 Insert tubes into plastic rack	65.0	17.5	35.0	<u>60.0</u>
4 Sort medicine boxes and parts	<u>62.5</u>	47.5	42.5	65.0
5 Classify colored blocks	<u>80.0</u>	55.0	67.5	87.5
6 Place objects into tray	<u>87.5</u>	67.5	70.0	90.0
7 Prepare coffee, long-horizon	52.5	0.0	7.5	<u>40.0</u>
Avg.	71.4	36.8	44.3	<u>68.9</u>

Statistical reliability. Table 9 reports 95% Wilson intervals for the real-robot success rates. Each task has 40 trials, so the category means of Table 6 equal the success rates pooled over the 160 Mobile Manipulation trials and the 280 Manipulation trials of Table 7. On Mobile Manipulation, the intervals of UniWAM-Scaled and $\pi_{0.5}$ do not overlap. On Manipulation, the intervals of UniWAM-Scaled and $\pi_{0.5}$ overlap, as do those of UniWAM-Base and $\pi_{0.5}$ on Mobile Manipulation. Hence, this comparison shows no clear difference between these pairs, while UniWAM-Base still docks more accurately than $\pi_{0.5}$. The intervals of UniWAM-Base and its real-robot ablations also overlap, so the ablation conclusions rest on the error metrics together with the success rates. These intervals treat trials as independent and ignore the shared initial configurations, so they are conservative for paired comparisons.

Table 8 Zero-shot real-robot MAP Navigation of models trained only on MAP-Data, averaged over the Cobot Magic and the AgiBot G2. The bottom rows are models trained on real demonstrations, taken from Table 6.

Model	Training data	E_{pos} (cm)	E_{head} (°)
$\pi_{0.5}$	MAP-Data only	55.1	11.4
GR00T N1.7	MAP-Data only	35.0	12.9
FastWAM	MAP-Data only	30.8	8.61
UniWAM-Sim	MAP-Data only	18.7	6.34
<i>Reference: trained on real demonstrations</i>			
$\pi_{0.5}$	real task data	23.3	5.15
UniWAM-Base	real task data	15.1	4.91

Table 9 Real-robot success rates with 95% Wilson intervals, pooled over 160 Mobile Manipulation trials and 280 Manipulation trials per method, matching the category means of Table 6. Brackets give the interval bounds.

Model	Mobile Manipulation SR (%)	Manipulation SR (%)
$\pi_{0.5}$	62.5 [54.8, 69.6]	71.4 [65.9, 76.4]
GR00T N1.7	39.4 [32.1, 47.1]	36.8 [31.4, 42.6]
UniWAM-Base	63.1 [55.4, 70.2]	44.3 [38.6, 50.1]
UniWAM-Scaled	80.0 [73.1, 85.5]	68.9 [63.3, 74.1]
w/o navigation auxiliary supervision	56.9 [49.1, 64.3]	43.6 [37.9, 49.4]
w/o manipulation auxiliary supervision	58.8 [51.0, 66.1]	39.3 [33.7, 45.1]
w/o independent sampling	60.0 [52.3, 67.3]	41.8 [36.2, 47.6]

4.4 Comparison of Unified Architectures

Fig. 5 and Table 10 compare four ways of unifying navigation and manipulation. All variants share the video DiT initialization, training data, optimization budget, evaluation protocol, and image-plane auxiliary targets, and they are evaluated on the four Mobile Manipulation tasks with the UniWAM-Base data. They differ only in how the two behaviors are represented, processed, and sampled, which isolates the effect of the stream design.

(a) Unified Action Tokens. Following $\pi_{0.5}$ [3], GR00T N1.7 [2], and FastWAM [41], each action token contains both the navigation and manipulation dimensions, and one action DiT predicts the combined command.

(b) Separate Action Experts. Navigation and manipulation use separate action DiTs that share the video DiT, and each action DiT predicts the command of its own behavior. This variant represents behavior-specific action experts in general and is implemented within our framework, not as a reimplementaion of a specific prior model. It therefore has more parameters than the mixed-stream design, which uses one action DiT.

(c) Sequence-Concatenated Streams. This is a controlled variant, not a reimplementaion of a prior method. Navigation and manipulation action sequences are concatenated into one token sequence and processed by one action DiT, so the two behaviors occupy different positions of one row and attend to each other.

(d) Mixed-Stream. In the proposed design, the two behaviors keep separate visual inputs, action encoders, and output heads, share the video and action DiTs, and occupy separate batch rows.

The first three variants require navigation and manipulation targets in every sample, so they are trained with joint sampling. We evaluate the mixed-stream design with both joint and independent sampling to separate the effect of the architecture from that of sampling, since this is the only change between them.

Architecture. Under joint sampling, the mixed-stream design obtains the lowest docking errors, 18.6 cm and 5.87°, and ties Sequence-Concatenated Streams in success at 60.0%. Compared with Unified Action Tokens, it reduces position and heading errors by 15.5% and 21.6% and improves success by 13.1 percentage points. Compared with Separate Action Experts, it lowers position and heading errors by 5.6% and 3.1% and raises success by 5.6 percentage points, despite a single shared action DiT. Relative to Sequence-Concatenated Streams, it reduces position and heading errors by 15.8% and 13.4% at the same success rate of 60.0%, so separating the streams into batch rows helps docking.

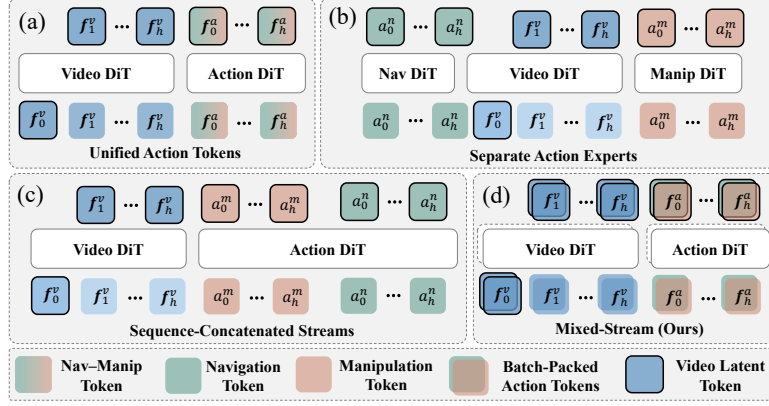


Figure 5 Unified mobile-manipulation architectures compared in Table 10: (a) unified action tokens, (b) separate navigation and manipulation action DiTs, (c) streams concatenated along the sequence axis, and (d) Mixed-Stream (ours), with streams in separate batch rows of shared video and action DiTs and independent stream sampling.

Table 10 Architecture and sampling comparison on the four Mobile Manipulation tasks. (a) to (c) use joint sampling. (d) uses joint or independent sampling, and the latter is UniWAM-Base as reported in Table 6.

Metric	Alternative architectures			(d) Mixed-stream	
	(a) Unified tokens	(b) Sep. experts	(c) Seq. concat.	Joint	Indep.
E_{pos} (cm) ↓	22.0	19.7	22.1	18.6	17.1
E_{head} (°) ↓	7.49	6.06	6.78	5.87	5.30
SR (%) ↑	46.9	54.4	60.0	60.0	63.1

Sampling. In the mixed-stream design, independent sampling further reduces the errors from 18.6 cm and 5.87° to 17.1 cm and 5.30° and increases success from 60.0% to 63.1%. Both design choices reduce docking errors, and together they give the highest success of all five variants in the table.

4.5 Ablation Studies

Each ablation changes one factor of UniWAM-Base with the data, training budget, and evaluation protocol fixed. Table 11 reports the MAP-Bench ablations, and Table 12 reports the real-robot ablations.

Pose targets. Replacing the relative pose target ($\Delta x, \Delta y, \Delta \theta$) with velocity commands (v, ω) increases the Overall errors from 12.5 cm and 3.24° to 16.3 cm and 4.17°. The position error rises only from 7.71 to 8.30 cm at short range but from 21.0 to 30.9 cm at long range. The degradation grows with distance, consistent with velocity errors that accumulate over time and with the bimodal velocity labels discussed in Sec. 3.2.1.

Causal mask. Removing the causal mask increases the Overall errors to 16.0 cm and 4.07° and the long-range position error from 21.0 to 29.0 cm. The mask lets later actions condition on the clean committed prefix without leaking information from later noisy actions backward, which matters most over long approaches with many successive chunks.

Navigation auxiliary targets. The two navigation targets have different distance-dependent effects. Removing the target box barely changes the short-range errors but raises the long-range errors from 21.0 cm and 4.29° to 29.5 cm and 5.14°, suggesting that the box helps localize distant, small targets. Removing the projected MAP, which marks where the base should stop, instead raises the short-range errors more than removing the box, from 7.71 cm and 2.37° to 8.05 cm and 2.63°. Removing both raises the Overall errors to 16.7 cm and 5.37°, above those of GR00T N1.7 and $\pi_{0.5}$, which suggests that the two targets have complementary effects on docking.

Auxiliary supervision on real robots. Removing navigation auxiliary supervision changes MAP Navigation from 15.1 cm and 4.91° to 17.6 cm and 6.84° and lowers Mobile Manipulation success from 63.1% to 56.9%. Removing manipulation auxiliary supervision leaves navigation close to the full model but lowers Manipulation success from

Table 11 Ablations of UniWAM-Base on MAP-Bench. The first row is the full model.

Variant	E_{pos} (cm) ↓			Overall		E_{head} (°) ↓			Overall
	Short	Mid	Long			Short	Mid	Long	
UniWAM-Base	7.71	8.85	21.0	12.5		2.37	3.06	4.29	3.24
Velocity actions (v, ω)	8.30	9.57	30.9	16.3		3.03	3.29	6.19	4.17
w/o causal mask	8.83	10.1	29.0	16.0		3.21	3.50	5.49	4.07
w/o target box	7.74	9.29	29.5	15.5		2.43	3.02	5.14	3.53
w/o image-plane MAP	8.05	8.40	22.5	13.0		2.63	2.82	4.72	3.39
w/o both auxiliary targets	9.31	11.6	29.1	16.7		4.45	4.91	6.76	5.37

Table 12 Ablations of UniWAM-Base on real robots. The first row is the full model.

Variant	MAP Navigation			Mobile Manipulation				Manipulation
	E_{pos} (cm) ↓	E_{head} (°) ↓		E_{pos} (cm) ↓	E_{head} (°) ↓	SR (%) ↑		SR (%) ↑
UniWAM-Base	15.1	4.91		17.1	5.30	63.1		44.3
w/o navigation auxiliary supervision	17.6	6.84		19.9	7.21	56.9		43.6
w/o manipulation auxiliary supervision	15.4	5.18		17.8	5.61	58.8		39.3
w/o independent sampling	16.8	5.62		18.6	5.87	60.0		41.8

44.3% to 39.3% and Mobile Manipulation success from 63.1% to 58.8%. Each auxiliary target has its largest effect on its own stream, and Figs. 8 and 9 visualize these predictions.

Independent sampling. Removing independent sampling worsens all six real-robot metrics, raising the MAP Navigation errors to 16.8 cm and 5.62° and lowering Mobile Manipulation success to 60.0%. Independent sampling thus improves both the navigation and the manipulation stream without any extra data.

Learnable tokens. Table 13 varies the number of learnable tokens per token bank over $\{0, 4, 8\}$, using the same count for all four banks. Removing the tokens degrades all six metrics. The MAP Navigation errors increase from 15.1 cm and 4.91° to 16.2 cm and 5.38°, and Mobile Manipulation success drops from 63.1% to 61.9%. Explicit token identity thus helps the shared DiTs distinguish the two streams. Increasing the count to eight changes each success rate by at most one trial, from 63.1% to 63.8% and from 44.3% to 44.6%, and gives mixed error changes, so we use four tokens per bank.

Manipulation coordinate frame. Table 14 compares camera-frame and robot-base end-effector targets under multi-embodiment co-training. Camera-frame targets improve success on each reported embodiment and task group and raise the unweighted mean from 48.5% to 55.2%. Following the alignment principle of Qwen-RobotManip [40], expressing end-effector motion in the current camera frame makes visually similar motions numerically similar across embodiments. The same visual change in the egocentric image then corresponds to similar action targets, regardless of the base frame or arm geometry of each robot, which lets data from different embodiments reinforce one another. This comparison covers co-training on these embodiments and does not evaluate transfer to unseen robots.

4.6 Qualitative Results

Trajectories. Fig. 7 compares the trajectories of all methods on 24 randomly selected MAP-Bench scenes with initial distances from 1.6 to 4.2 m. The two VLN baselines show the typical failures of semantic navigation. InternVLA-N1 heads to a wrong region in many displayed scenes, for example in panels a to g. Uni-NaVid often reaches the vicinity of the target but stops beside it, as in panels b, f, l, and n. The VLA and WAM baselines reach the target region in most displayed scenes but show visible path deviations or terminal offsets, such as FastWAM in panels j and u and $\pi_{0.5}$ in panel r. Both UniWAM variants follow the reference path closely and end at the MAP in most displayed scenes. The contrast is largest in the last two rows, where all initial distances are at least 3.5 m. There, the VLN baselines fail in most displayed scenes and the other baselines deviate more often, whereas UniWAM remains close to the reference path, in agreement with the long-range gains of both UniWAM variants in Table 4.

Auxiliary predictions. Fig. 8 overlays the target boxes and projected MAP positions that UniWAM predicts over

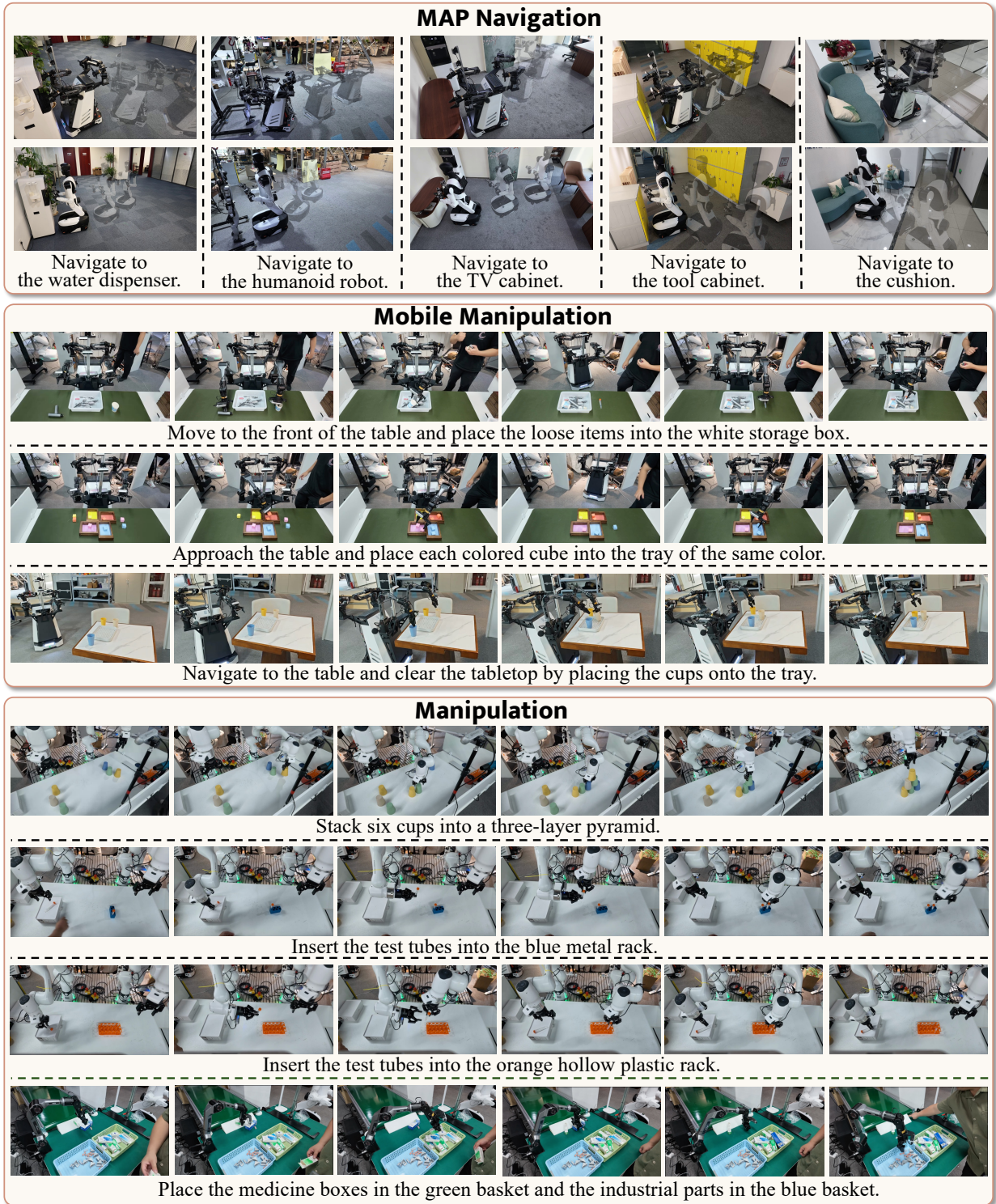


Figure 6 Real-robot executions of UniWAM. A single UniWAM model performs MAP Navigation, Mobile Manipulation, and Manipulation across four embodiments. The MAP Navigation examples dock at targets of different categories and sizes, the Mobile Manipulation examples show the transition from approach to manipulation, and the Manipulation examples show object sorting, test-tube insertion, and cup stacking.

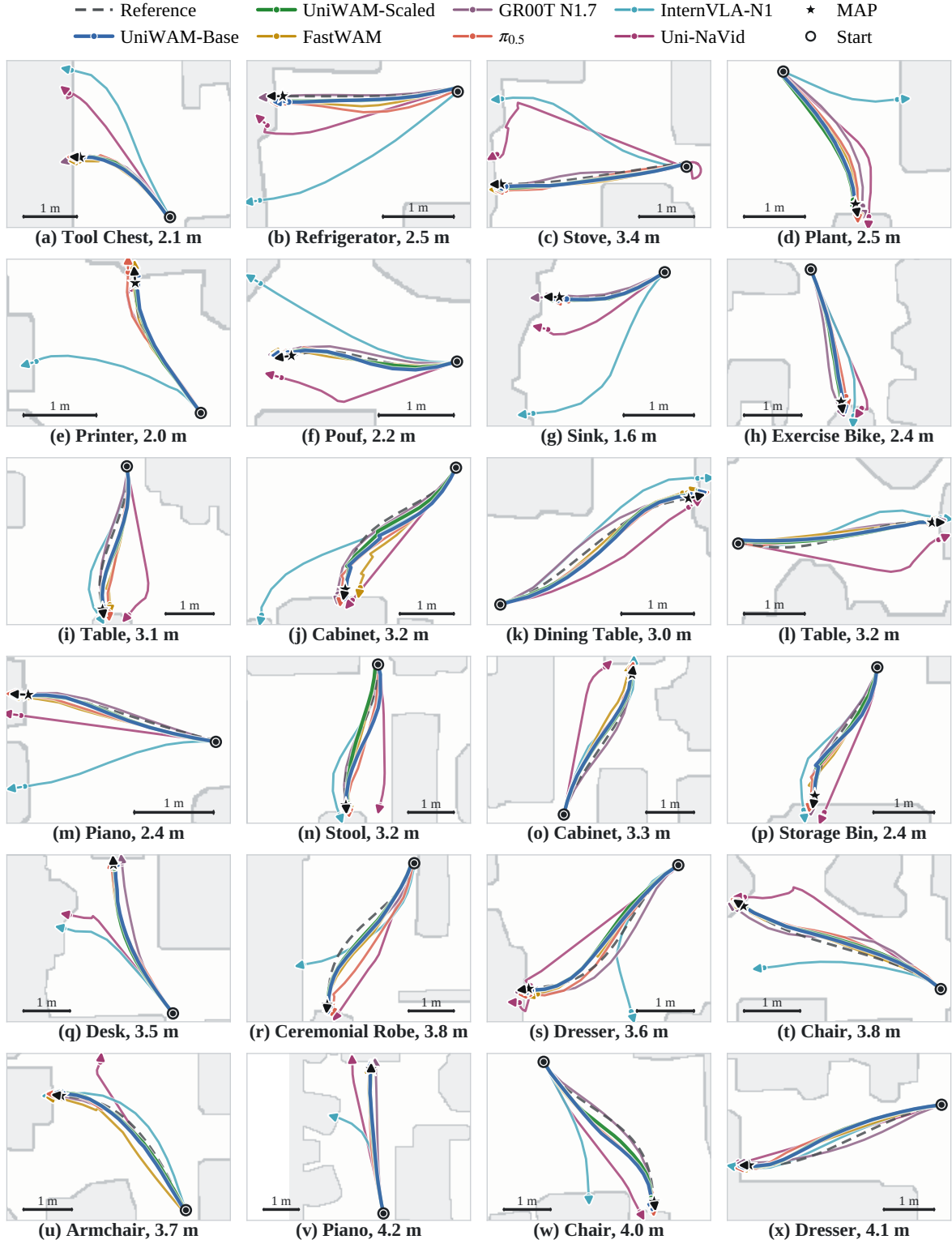


Figure 7 MAP-Bench trajectories of all evaluated methods on 24 randomly selected scenes. Subcaptions give the target category and initial distance. Stars mark the reference MAP, circles the start pose, and arrowheads the terminal heading. In the last two rows, every episode starts at least 3.5 m from the MAP, which falls in the long range.

Table 13 Real-robot results for the number of learnable tokens per bank. † marks the default.

Tokens per bank	MAP Navigation		Mobile Manipulation			Manipulation
	E_{pos} (cm) ↓	E_{head} (°) ↓	E_{pos} (cm) ↓	E_{head} (°) ↓	SR (%) ↑	SR (%) ↑
0	16.2	5.38	17.8	5.48	61.9	43.6
4, ours [†]	15.1	4.91	17.1	5.30	63.1	44.3
8	14.8	5.02	17.3	5.24	63.8	44.6

Table 14 Manipulation target frame in UniWAM-Base under multi-embodiment co-training. Values are success rates in percent. Cobot Magic uses the Mobile Manipulation tasks, the dual Franka uses Manipulation tasks 1 to 3 and 7, and the AgileX single arm uses Manipulation task 6. Mean is the unweighted mean over the three reported embodiment and task groups.

Target frame	Cobot Magic	Dual Franka	AgileX single arm	Mean
Robot base	55.0	28.1	62.5	48.5
Camera, ours	63.1	32.5	70.0	55.2

consecutive frames of MAP-Bench episodes. The boxes tightly enclose the target across object categories, scenes, and viewing distances, from large targets that fill much of the view, such as the stove and the door, to smaller targets such as the chairs. The boxes from consecutive frames nearly coincide, so the predictions remain stable over time rather than jittering from frame to frame. The projected MAP lies on the floor directly in front of the manipulation-facing side of the target, at the point where the base should stop. These observations match the ablation in Sec. 4.5. The box keeps the target localized throughout the approach, and the MAP point marks the terminal pose.

End-effector predictions. Fig. 9 examines whether the predicted 2D end-effector tracks agree with the executed 3D actions. Each curve is the track predicted for one arm in the previous action chunk, and each circle marks the actual end-effector position after the robot executes that chunk. Across both arms, several tasks, and two embodiments, the circles lie on the predicted tracks at the grippers. In other words, the image-plane tracks agree with where the camera-frame actions actually move the end effectors. The manipulation stream has therefore learned a consistent mapping from its 3D end-effector actions to their pixel locations in the main camera view of each embodiment.

4.7 Failure Cases

Grasping between adjacent objects. Fig. 10 shows a failure of UniWAM in the color-sorting task. A pink and a blue cube stand next to each other on the table, and either may be picked next. Instead of committing to one of them, the gripper descends toward the point between the two cubes and closes on the gap. The demonstrations contain grasps of either cube, so the action distribution has two nearby modes. When the modes are close, the deterministic ten-step sampler can produce an action between them rather than on one of them. We observed this failure when candidate objects were close together, and specifying the target object in the instruction may reduce the ambiguity.

Residual base motion. During the manipulation stage, the navigation stream predicts motions that are close to but not exactly zero, which can move the base slightly. The near-zero rule suppresses it for all methods.

**Figure 10** Failure case in the color-sorting task. From left to right, a pink and a blue cube stand next to each other, and the gripper descends toward the point between them and closes on the empty gap instead of a cube.

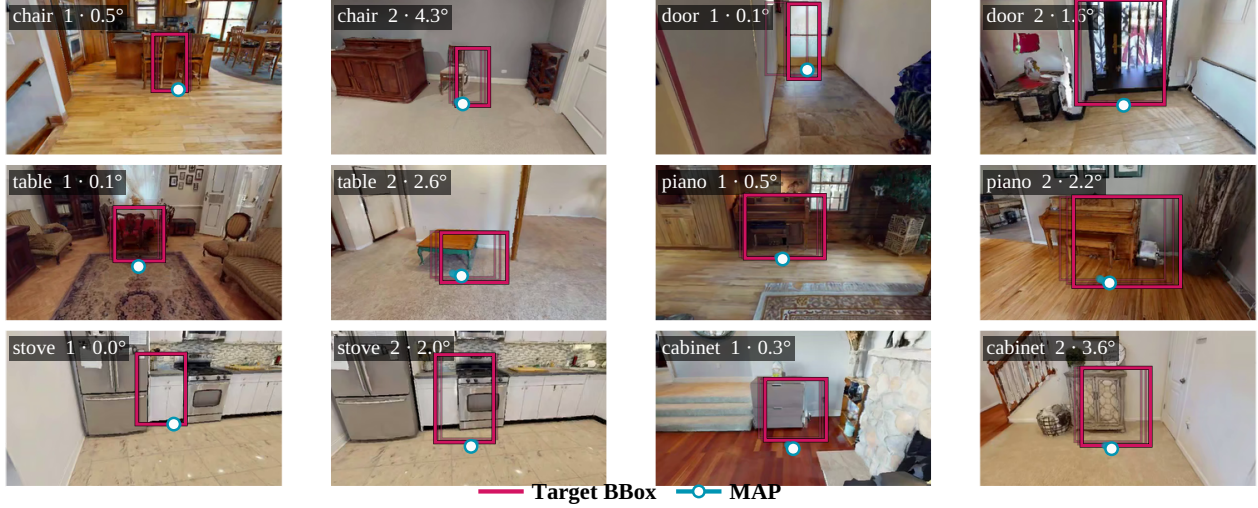


Figure 8 Navigation auxiliary predictions of UniWAM on MAP-Bench, overlaid over consecutive frames of each episode. Magenta boxes are the predicted target boxes, and cyan points are the predicted MAP positions projected into the image, both predicted jointly with the actions by the navigation stream of UniWAM at every step of the approach.



Figure 9 Alignment between predicted 2D end-effector tracks and executed 3D actions on several tasks and two embodiments. Curves show the tracks of the two arms predicted in the previous action chunk, and circles mark the positions reached after that chunk.

5 Conclusion

We presented UniWAM, a mixed-stream world-action model that treats navigation and manipulation as two independently sampled control streams over shared video and action DiTs, and the Manipulation Anchor Pose (MAP), an explicit target for manipulation-ready navigation supported by over 1.5 million automatically generated MAP-Data episodes. Independent sampling lets UniWAM learn directly from navigation-only and manipulation-only data, and image-plane auxiliary supervision grounds the actions of both streams in the egocentric image. On MAP-Bench, UniWAM trained

with MAP-Data reduces position and heading errors by 30.1% and 44.0% relative to the strongest baselines. On 24 real-robot tasks across four embodiments, a single model trained with additional data achieves the most accurate docking and the highest mobile-manipulation success while remaining competitive in manipulation.

Future work. We will conduct large-scale real-robot pretraining of the UniWAM architecture, combining diverse real-robot demonstrations with MAP-Data, and study how the mixed-stream design scales with robot data.

References

- [1] Hongzhe Bi, Hengkai Tan, Shenghao Xie, Zeyuan Wang, Shuhe Huang, Haitian Liu, Ruowen Zhao, Yao Feng, et al. Motus: A unified latent action world model. *arXiv preprint arXiv:2512.13030*, 2025.
- [2] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [3] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. In *Conference on Robot Learning*, 2025.
- [4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, et al. π_0 : A vision-language-action flow model for general robot control. In *Robotics: Science and Systems*, 2025.
- [5] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, et al. RT-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems*, 2023.
- [6] Kaixin Chai, Hyunjun Lee, and Joseph J. Lim. N2M: Bridging navigation and manipulation by learning pose preference from rollout. *arXiv preprint arXiv:2509.18671*, 2025.
- [7] Guangyu Chen, Qiwei Liang, Shaolong Zhu, Tianxing Chen, Zikuan Xiao, Yifan Xie, Lingfeng Zhang, Ping Luo, Renjing Xu, and Wenbo Ding. MoPA: Coordinated mobile manipulation via subsystem-specific perception alignment. *arXiv preprint arXiv:2609.12081*, 2026.
- [8] Jizhuo Chen, Diwen Liu, Jiaming Wang, and Harold Soh. GBPP: Grasp-aware base placement prediction for robots via two-stage learning. *arXiv preprint arXiv:2509.11594*, 2026.
- [9] Ronghan Chen, Yandan Yang, Zuojin Tang, Dongjie Huo, Tong Lin, Haoning Wu, Haoyun Liu, Yuzhi Chen, et al. ABot-M0.5: Unified mobility-and-manipulation world action model. *arXiv preprint arXiv:2607.00678*, 2026.
- [10] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.
- [11] Ria Doshi, Homer Rich Walke, Oier Mees, Sudeep Dasari, and Sergey Levine. Scaling cross-embodied learning: One policy for manipulation, navigation, locomotion and aviation. In *Conference on Robot Learning*, 2024.
- [12] Zehua Fan, Junjie He, Wenxuan Song, Xi Wang, Wenqi Lyu, Linge Zhao, Fuhao Li, Zihan You, et al. MobileWAM: Bridging world action models to mobile manipulation with chain-of-foresight. *arXiv preprint arXiv:2608.04657*, 2026.
- [13] Zipeng Fu, Tony Z. Zhao, and Chelsea Finn. Mobile ALOHA: Learning bimanual mobile manipulation with low-cost whole-body teleoperation. In *Conference on Robot Learning*, 2024.
- [14] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [15] InternNav Team. InternVLA-N1: An open dual-system navigation foundation model with learned latent plans. Technical report, 2025. URL <https://internrobotics.github.io/internvla-n1.github.io/>.
- [16] Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, et al. Cosmos policy: Fine-tuning video models for visuomotor control and planning. *arXiv preprint arXiv:2601.16163*, 2026.
- [17] Lin Li, Qihang Zhang, Yiming Luo, Shuai Yang, Ruilin Wang, Fei Han, Mingrui Yu, Zelin Gao, et al. Causal world modeling for robot control. *arXiv preprint arXiv:2601.21998*, 2026.
- [18] Weixin Liang, Lili Yu, Liang Luo, Srinivasan Iyer, Ning Dong, Chunting Zhou, et al. Mixture-of-transformers: A sparse and scalable architecture for multi-modal foundation models. *arXiv preprint arXiv:2411.04996*, 2024.

- [19] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=PqvMRDCJT9t>.
- [20] Bingchen Miao, Rong Wei, Zhiqi Ge, Xiaoquan Sun, Shiqi Gao, Jingzhe Zhu, Renhan Wang, Siliang Tang, Jun Xiao, Rui Tang, and Juncheng Li. Towards physically executable 3D gaussian for embodied navigation. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [21] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.
- [22] Jonas Pai, Liam Achenbach, Victoriano Montesinos, Benedek Forrai, Oier Mees, and Elvis Nava. mimic-video: Video-action models for generalizable robot control beyond VLAs. *arXiv preprint arXiv:2512.15692*, 2025.
- [23] Physical Intelligence, Bo Ai, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, et al. $\pi_{0.7}$: A steerable generalist robotic foundation model with emergent capabilities. *arXiv preprint arXiv:2604.15483*, 2026.
- [24] Santhosh Kumar Ramakrishnan, Aaron Gokaslan, Erik Wijmans, Oleksandr Maksymets, Alexander Clegg, John M. Turner, Eric Undersander, Wojciech Galuba, Andrew Westbury, Angel X. Chang, et al. Habitat-matterport 3d dataset (HM3D): 1000 large-scale 3d environments for embodied AI. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- [25] Ruisen Tu, Arth Shukla, Sohyun Yoo, Xuanlin Li, Junxi Li, Jianwen Xie, Hao Su, and Zhuowen Tu. SG-VLA: Learning spatially-grounded vision-language-action models for mobile manipulation. *arXiv preprint arXiv:2603.22760*, 2026.
- [26] Nikolaus Vahrenkamp, Tamim Asfour, and Rüdiger Dillmann. Robot placement based on reachability inversion. In *IEEE International Conference on Robotics and Automation*, pages 1970–1975, 2013.
- [27] Wan Team, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.
- [28] Lirui Wang, Xinlei Chen, Jialiang Zhao, and Kaiming He. Scaling proprioceptive-visual learning with heterogeneous pre-trained transformers. In *Advances in Neural Information Processing Systems*, 2024.
- [29] Meng Wei, Chenyang Wan, Jiaqi Peng, Xiqian Yu, Yuqiang Yang, Delin Feng, Wenzhe Cai, Chenming Zhu, Tai Wang, Jiangmiao Pang, and Xihui Liu. Ground slow, move fast: A dual-system foundation model for generalizable vision-language navigation. In *International Conference on Learning Representations*, 2026.
- [30] Zhenyu Wu, Angyuan Ma, Xiuwei Xu, Hang Yin, Yinan Liang, Ziwei Wang, Jiwen Lu, and Haibin Yan. MoTo: A zero-shot plug-in interaction-aware navigation for general mobile manipulation. *arXiv preprint arXiv:2509.01658*, 2025.
- [31] Zhenyu Wu, Yuheng Zhou, Xiuwei Xu, Ziwei Wang, and Haibin Yan. MoManipVLA: Transferring vision-language-action models for general mobile manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.
- [32] Xiaomi Robotics Team, Jun Guo, Piaopiao Jin, Jason Li, Peiyan Li, Yingyan Li, Futeng Liu, Wanli Peng, et al. Xiaomi-Robotics-1: Scaling vision-language-action models with over 100k hours of real-world trajectories. *arXiv preprint arXiv:2607.15330*, 2026.
- [33] Xiaomeng Xu, Jisang Park, Han Zhang, Eric Cousineau, Aditya Bhat, Jose Barreiros, Dian Wang, Jeannette Bohg, and Shuran Song. HoMMI: Learning whole-body mobile manipulation from human demonstrations. In *Robotics: Science and Systems*, 2026.
- [34] Karmesh Yadav, Ram Ramrakhya, Santhosh Kumar Ramakrishnan, Theo Gervet, John Turner, Aaron Gokaslan, Noah Maestre, Angel X. Chang, Dhruv Batra, Manolis Savva, et al. Habitat-matterport 3d semantics dataset. *arXiv preprint arXiv:2210.05633*, 2022.
- [35] Donglin Yang, Haoran Chen, Xingyu Chen, Lixing Liu, Manyi Li, Changhe Tu, Ke Xu, Xiaojian Ma, and Si Liu. Learning panorama-aware VLA for mobile manipulation with whole-body teleoperation. *arXiv preprint arXiv:2608.02257*, 2026.
- [36] Jingyun Yang, Isabella Huang, Brandon Vu, Max Bajracharya, Rika Antonova, and Jeannette Bohg. Mobi- π : Mobilizing your robot learning policy. *arXiv preprint arXiv:2505.23692*, 2025.
- [37] Zheng Yang, Wenjie Zhang, Xiangyu Chen, Wenxuan Song, Xianpeng Wang, Yihang Kang, Jiawen Wen, Wen Chen, et al. DreamTrajectory: Trajectory-guided action generation with world model alignment for mobile manipulation. *arXiv preprint arXiv:2608.01381*, 2026.

- [38] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chungh Zhu, Jiannan Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.
- [39] Sriram Yenamandra, Arun Ramachandran, Karmesh Yadav, Austin Wang, Mukul Khanna, Theophile Gervet, Tsung-Yen Yang, Vidhi Jain, Alexander William Clegg, John Turner, et al. HomeRobot: Open-vocabulary mobile manipulation. *arXiv preprint arXiv:2306.11565*, 2023.
- [40] Haoqi Yuan, Zhixuan Liang, Anzhe Chen, Ye Wang, Haoyang Li, Pei Lin, Yiyang Huang, Zixing Lei, Tong Zhang, Jiazhao Zhang, et al. Qwen-RobotManip technical report: Alignment unlocks scale for robotic manipulation foundation models. *arXiv preprint arXiv:2606.17846*, 2026.
- [41] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026.
- [42] Jiazhao Zhang, Kunyu Wang, Shaoan Wang, Minghan Li, Haoran Liu, Songlin Wei, Zhongyuan Wang, Zhizheng Zhang, and He Wang. Uni-NaVid: A video-based vision-language-action model for unifying embodied navigation tasks. In *Robotics: Science and Systems*, 2025.
- [43] Pingrui Zhang, Xianqiang Gao, Yuhan Wu, Kehui Liu, Dong Wang, Zhigang Wang, Bin Zhao, Yan Ding, and Xuelong Li. MoMa-Kitchen: A 100k+ benchmark for affordance-grounded last-mile navigation in mobile manipulation. *arXiv preprint arXiv:2503.11081*, 2025.
- [44] Zhuofan Zhang, Tianxu Wang, Guoxi Zhang, Yixiong Lin, Xilin Wang, Hongming Xu, Qing Li, Song-Chun Zhu, and Lifeng Fan. UniLM-Nav: A unified framework for zero-shot last-mile navigation. *arXiv preprint arXiv:2607.06537*, 2026.
- [45] Ping Zhong, Manling Teng, Tao Wu, Bolei Chen, Jiazhi Xia, and Jianxin Wang. FloAff-Kitchen: Bridging navigation and manipulation via canonical and progressive floor affordance learning. *arXiv preprint arXiv:2607.24207*, 2026.
- [46] Yi Zhou, Connelly Barnes, Jingwan Lu, Jimei Yang, and Hao Li. On the continuity of rotation representations in neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5745–5753, 2019.
- [47] Lawrence Y. Zhu, Pranav Kuppli, Ryan Punamiya, Patcharapong Aphiwetsa, Dhruv Patel, Simar Kareer, Sehoon Ha, and Danfei Xu. EMMA: Scaling mobile manipulation via egocentric human data. *arXiv preprint arXiv:2509.04443*, 2025.
- [48] Xiangyu Zhu, Renjun Wu, Luzhou Ge, Jinyan Liu, and Xuesong Li. GeoHAT: Geometry-adaptive hybrid action transformer for mobile manipulation. *arXiv preprint arXiv:2606.13394*, 2026.